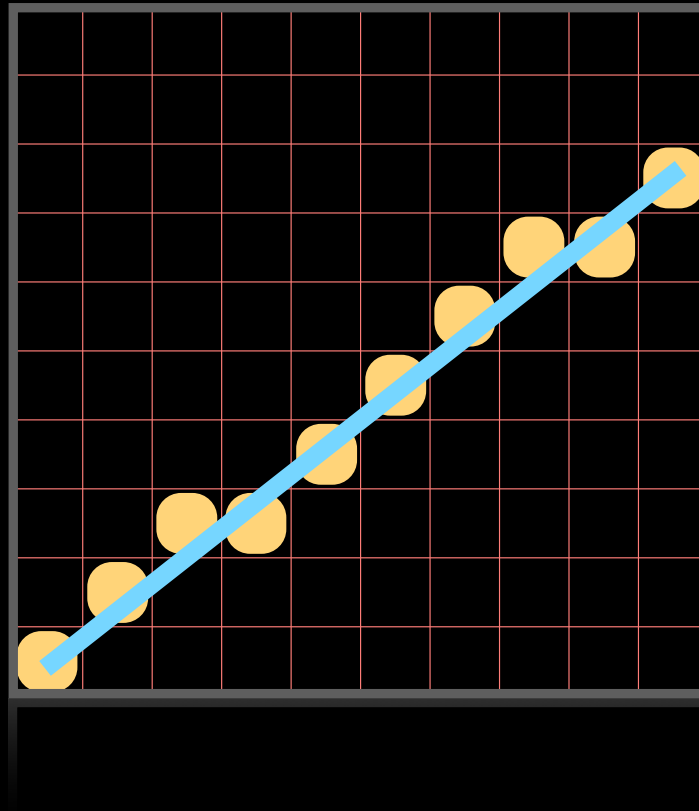
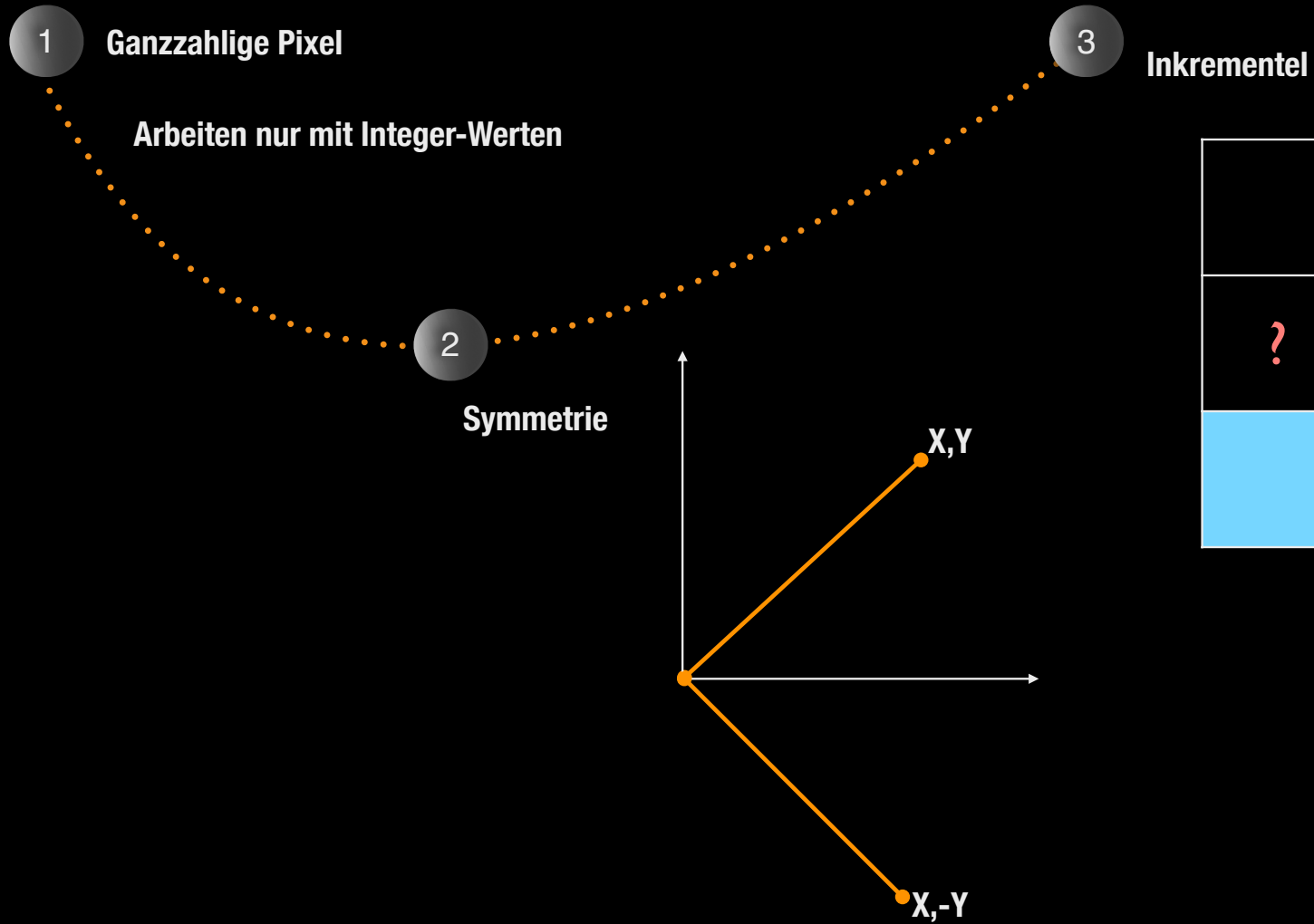


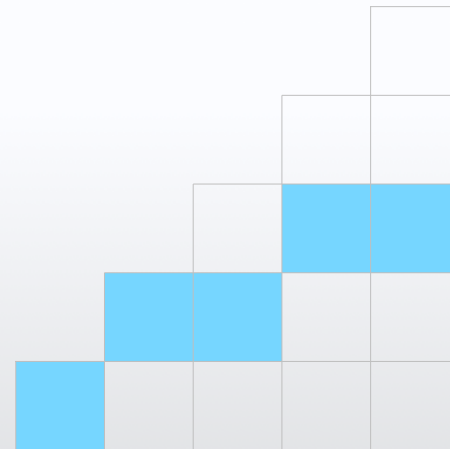
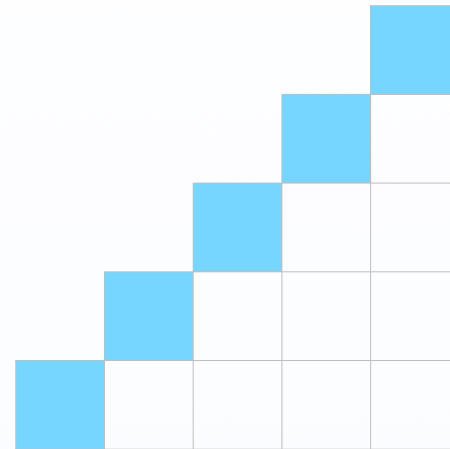




?	?	
	?	

Magische Bedingung

- Für den 1 Oktant
- Für jedes aufsteigende x wird ein Pixel gezeichnet
- Gehe entlang der Linie von Links nach Rechts
- Gehe dabei „eventuell“ ein Pixel hoch

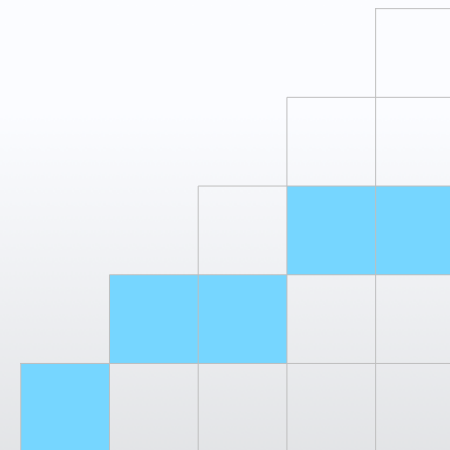
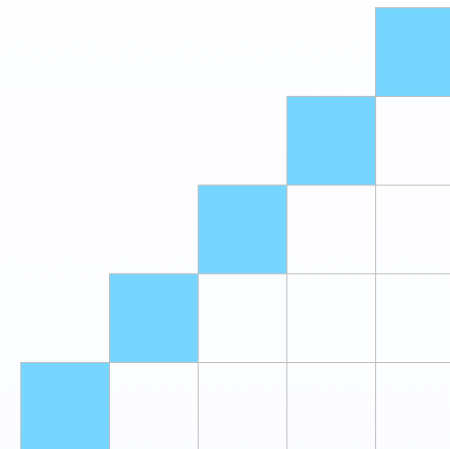


Magische Bedingung

- Für den 1 Oktant
- Für jedes aufsteigende x wird ein Pixel gezeichnet
- Gehe entlang der Linie von Links nach Rechts
- Gehe dabei „eventuell“ ein Pixel hoch

```
int y=y1;  
for(int x=x1; x <= x2 ; x++){  
    setPixel(x,y);  
    if (Bedingung)  
        y=y+1;  
    // sonst bleibt y  
}
```

wie sieht die Bedingung aus?



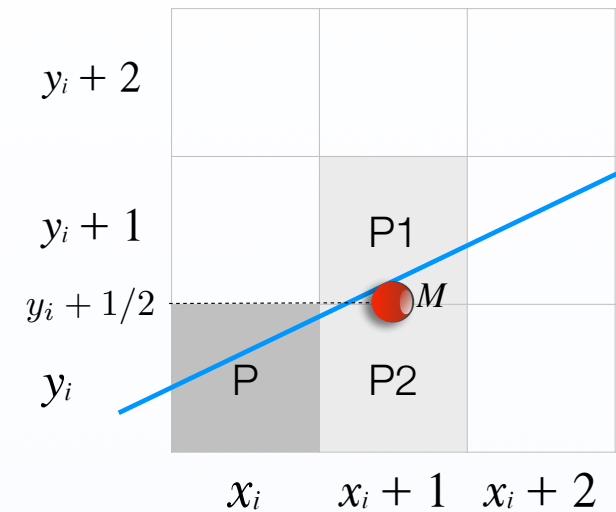
Midpoint-line Algorithmus „Bresenham '65“

- Bedingung: Punkt P gesetzt
- welcher Punkt ist der nächste?
- in Frage kommende Punkte P1, oder P2
- M = Mittelpunkt zwischen P1 & P2
- $M = (x_i + 1, y_i + 1/2)$
- IDEE:

- Koordinaten M in die Geradengleichung einsetzen

- Gerade ist überhalb M , nehme P1
- Gerade ist unterhalb M , nehme P2
- M auf der Gerade, nehme P1 oder P2

- Geradengleichung: $ax + by + c = 0$



Midpoint line Algorithmus

■ Geradengleichung $ax + by + c = 0$

■ Koordinaten von M einsetzen:

■ $a(x_i + 1) + b(y_i + 1/2) + c \geq 0$ } $y=y+1$
 ■ M ist im oberen Halbraum
 ■ Linie über M ; Wähle P1

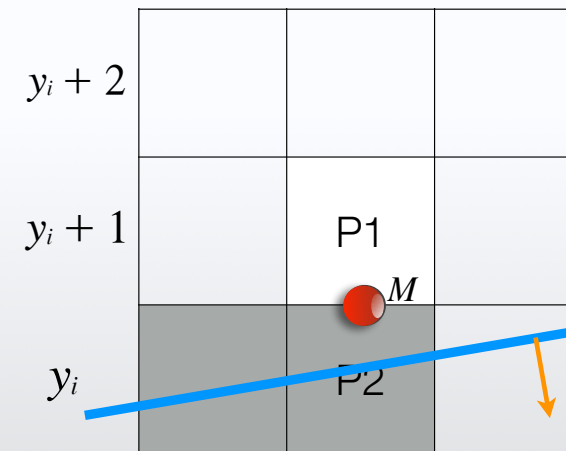
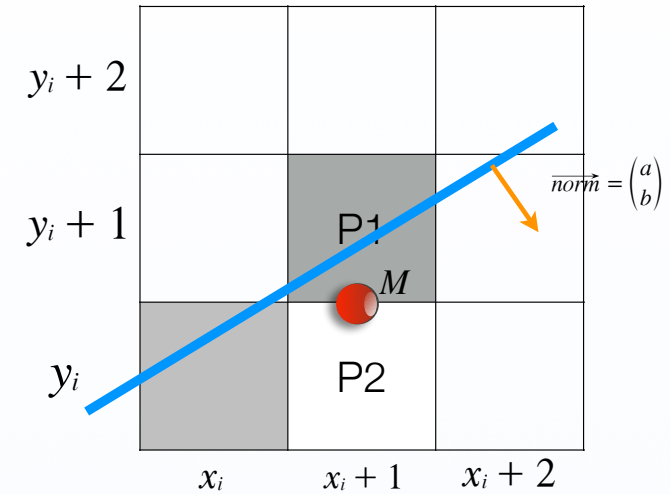
■ $a(x_i + 1) + b(y_i + 1/2) + c < 0$ } $y=y$
 ■ M ist im unteren Halbraum
 ■ Linie unter M ; Wähle P2

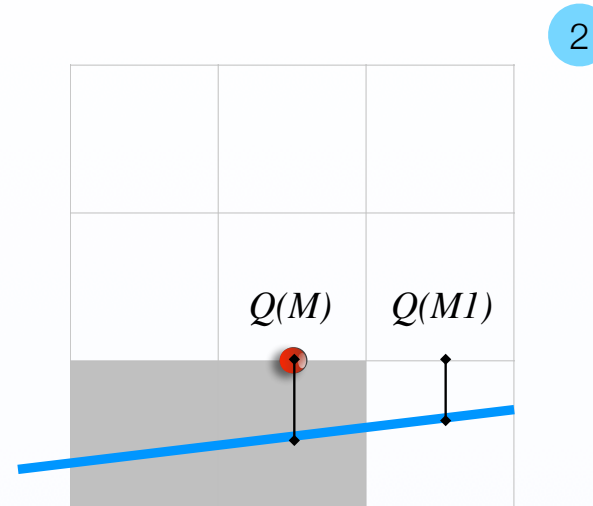
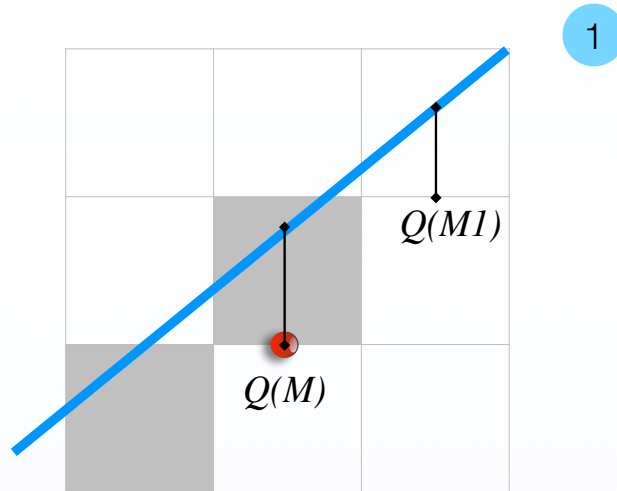
Mit

$$a = \Delta y$$

$$b = -\Delta x$$

$$c = \Delta x \cdot n$$





Geradengleichung an der Stelle $M :: Q(M)$ berechnen
Geradengleichung an der Stelle $M1 :: Q(M1)$ berechnen

Kann $Q(M1)$ aus $Q(M)$ gebildet werden?
Was unterscheidet die Werte ?

$$Q(M1) - Q(M)$$

Bresenham (inkrement) 1

- P1 ist gewählt mit M

- Geradengleichung ist positiv

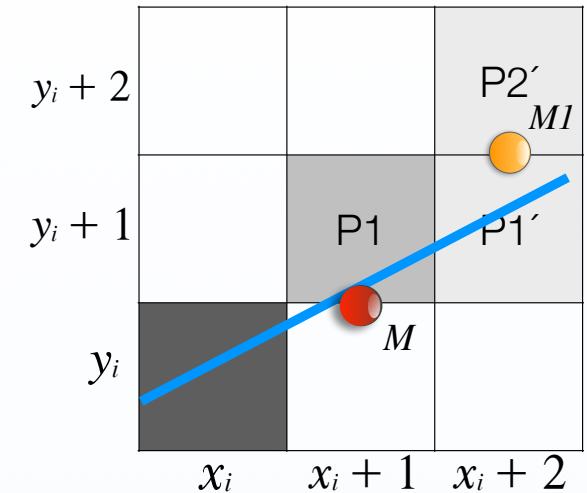
$$Q(M) \quad a(x_i + 1) + b(y_i + 1/2) + c \geq 0$$

- Dann ist $M1$ der nächste Mittelpunkt
- Geradengleichung mit $M1$ -Koordinaten.

$$Q(M1) \quad a(x_i + 2) + b(y_i + 3/2) + c \geq 0$$

- Differenz: $Q(M1) - Q(M) = a + b$

c und damit n fallen weg! ($c = \Delta x \cdot n$)



Bresenham (inkrement)

Entscheidungsvariable Q inkrementel berechnen

$$Q(M1) - Q(M) = a + b$$

Daraus ergibt sich :

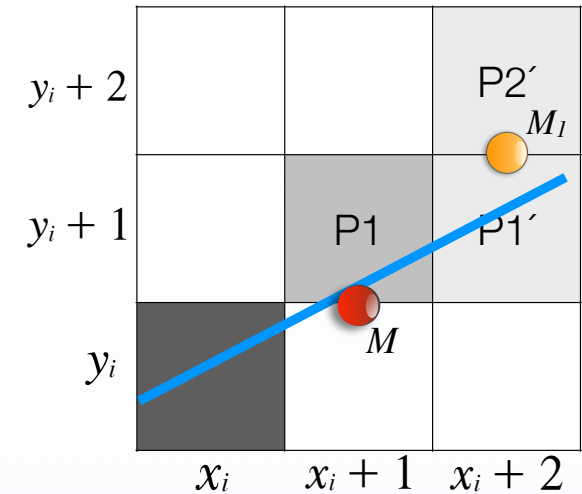
$$Q(M1) = Q(M) + a + b$$

mit $a = \Delta y$, $b = -\Delta x$

$$Q(M1) = Q(M) + \Delta y - \Delta x$$

In Code

```
if Q (positiv) {  
    select P1;  
    ...  
    Q= Q + Dy - Dx;  
}
```



Bresenham (inkrement) 2

- P2 ist gewählt mit M

- Geradengleichung ist negativ

$$Q(M) \quad a(x_i + 1) + b(y_i + 1/2) + c < 0$$

- Dann ist M_I der nächste Mittelpunkt
- Geradengleichung mit M_I -Koordinaten.

$$Q(M_I) \quad a(x_i + 2) + b(y_i + 1/2) + c \quad ? \quad 0$$

- Differenz: $Q(M_I) - Q(M) = a$

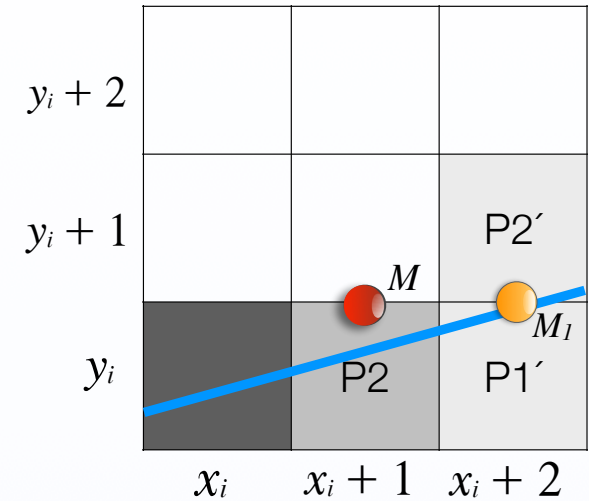
- mit $a = \Delta y$

- Aktualisiere Q

- $Q(M_I) = Q(M) + \Delta y$



```
if Q (negativ) {  
    select P2;  
    ...  
    Q= Q+ Dy;  
}
```

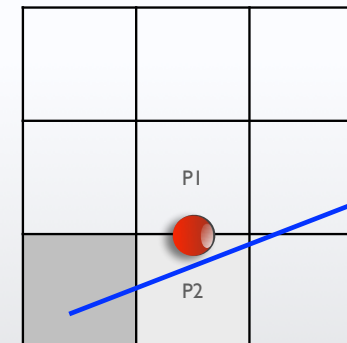
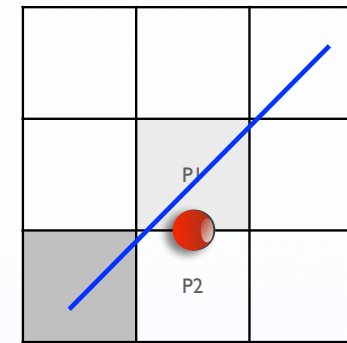


Algorithmus

- Setze den ersten Pixel
- Berechne Entscheidungsvariable Q für aktuelles Pixel
- Wiederhole bis Endpunkt erreicht ist

```
■ if (Q Positiv oder Null)
    ■ nächstes Pixel ist P1(oben);
    ■ x++; y++;
    ■ Q aktualisieren:  $Q = Q + \Delta y - \Delta x$ 

■ else (Q negativ)
    ■ nächstes Pixel ist P2 (unten);
    ■ x++; // y bleibt const
    ■ Q aktualisieren:  $Q = Q + \Delta y$ 
■ Pixel zeichnen
```



Mit welchem Initial-Wert für Q_{init} anfangen ?

Anfangswert für Q

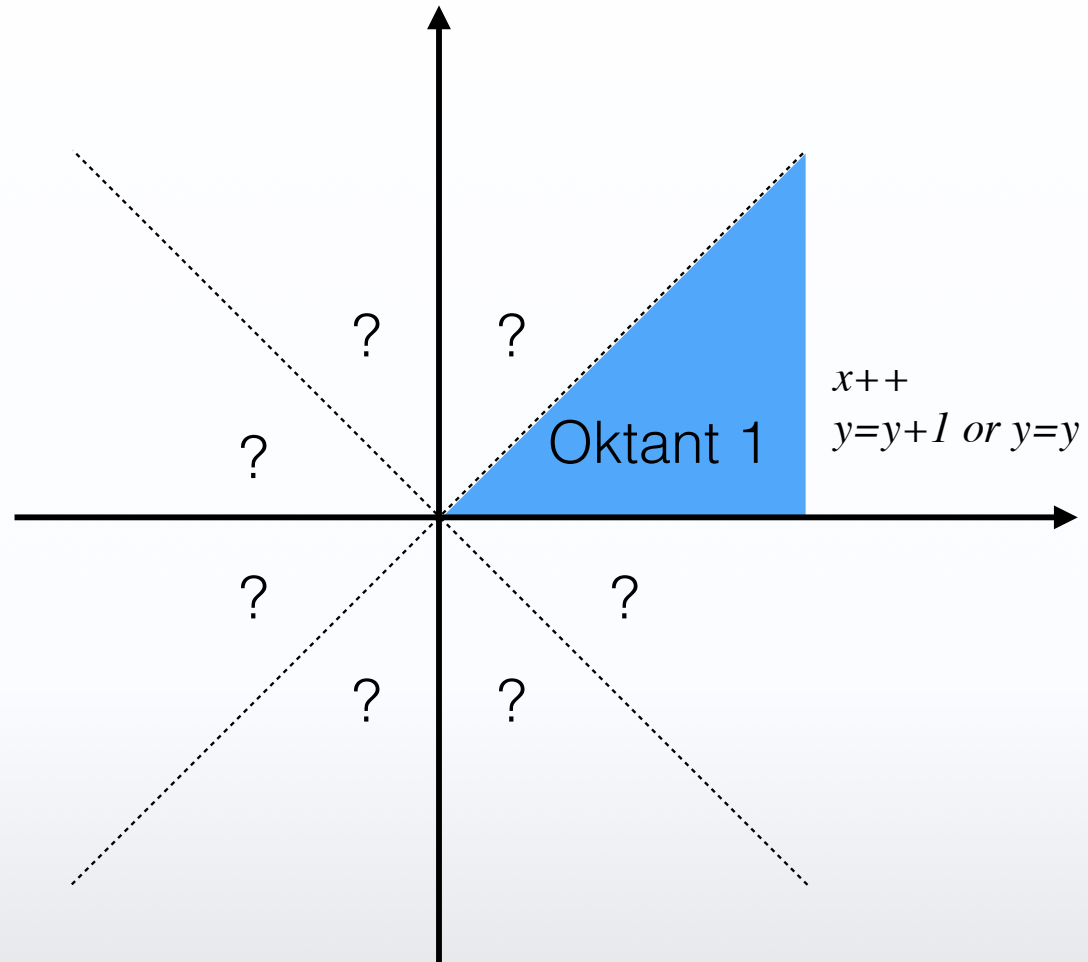
- Initialwert ist immer noch ein float Wert : $Q = a + \frac{1}{2}b$
- Da uns nur Vorzeichen interessiert : $Q = 2a + b$

```
void draw_line(int x1, int y1, int x2, int y2){

    int y=y1
    int a = y2-y1;          // delta y
    int b = -(x2-x1);       // - delta x
    int Q_init= 2*a+b;

    for(int x=x1; x<=x2 ; x++) {
        setPixel(x,y);
        if (Q<0)
            Q=Q+2*a;
        else {
            Q=Q+2*(a+b);
            y++;
        }
    }
}
```


Was ist mit den anderen Oktanten

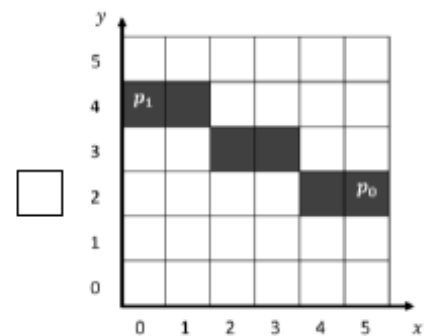
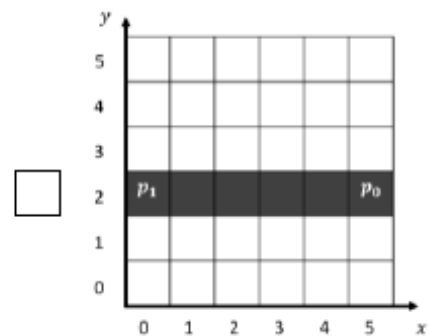
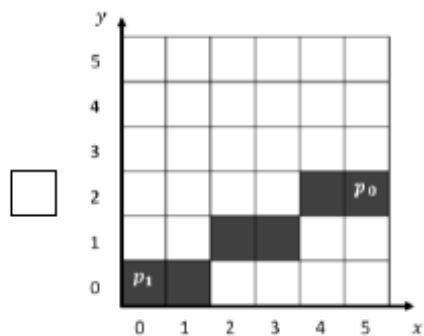
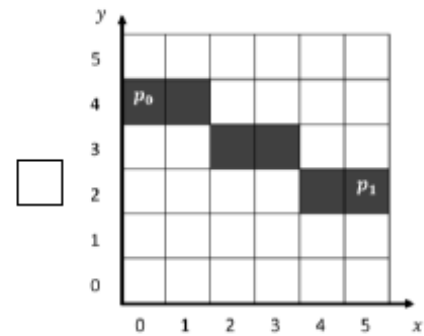
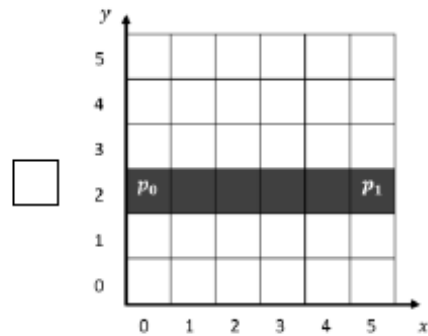
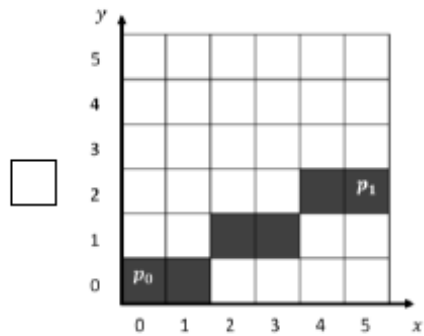


1. Bresenham

1.1. Gegeben sei folgender Quellcode der Funktion *drawBresenhamLine()*

```
1  function drawBresenhamLine(x0, y0, x1, y1) {
2      var deltaX = Math.abs(x1 - x0);
3      var deltaY = Math.abs(y1 - y0);
4
5      var Q = 2 * deltaY - deltaX;
6      var Qinc_a = 2 * (deltaY - deltaX);
7      var Qinc_b = 2 * deltaY;
8
9      var y = y0;
10
11     for( var x = x0; x <= x1; x++ ) {
12         setPixel( x, y );
13
14         if ( Q < 0 ) {
15             Q += Qinc_b;
16         }
17         else {
18             Q += Qinc_a;
19             y--;
20         }
21     }
22 }
```

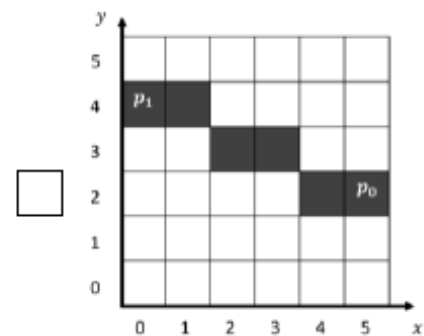
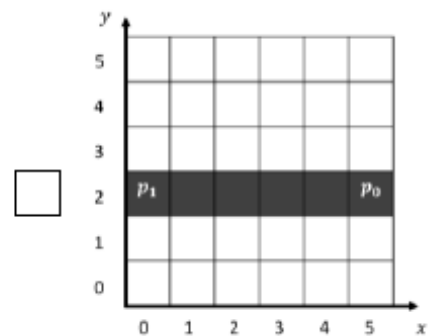
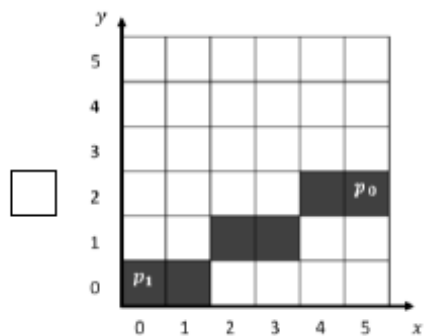
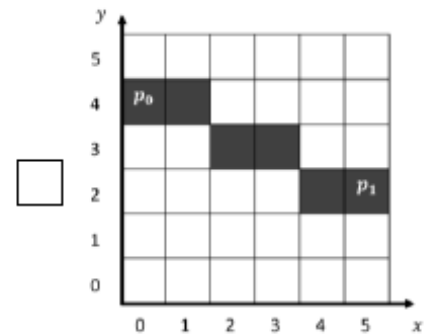
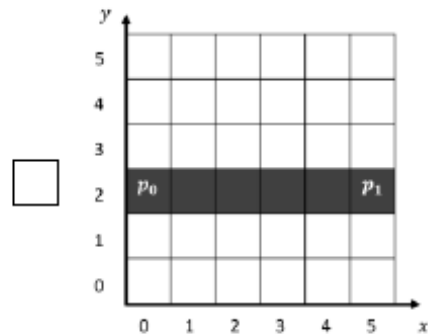
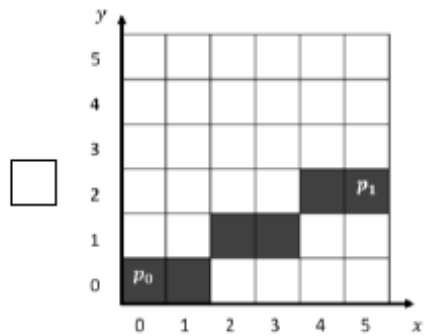
Es gelte $p_0 = (x_0, y_0)$, $p_1 = (x_1, y_1)$. Welche der folgenden Linien können mit obiger Funktion gezeichnet werden? (Mehrere Antworten sind möglich)



1.2. Der Code in den Zeilen 2 und 3 wird nun gemäß nachfolgenden Zeilen ersetzt. Was bewirkt diese Änderung?

```
2 | var deltaX = x1 - x0;
3 | var deltaY = y1 - y0;
```

Es gelte $p_0 = (x_0, y_0)$, $p_1 = (x_1, y_1)$. Welche der folgenden Linien können mit obiger Funktion gezeichnet werden? (Mehrere Antworten sind möglich)

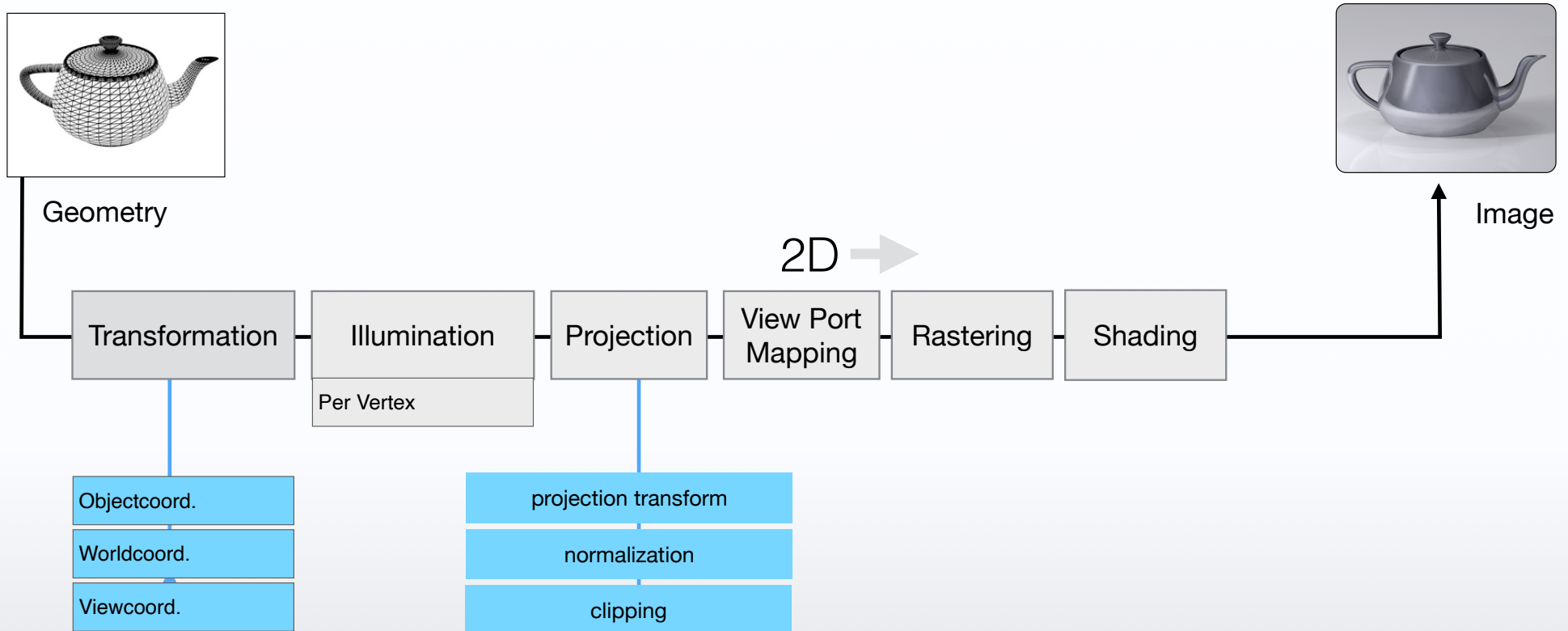


1.2. Der Code in den Zeilen 2 und 3 wird nun gemäß nachfolgenden Zeilen ersetzt. Was bewirkt diese Änderung?

```
2 | var deltaX = x1 - x0;
3 | var deltaY = y1 - y0;
```

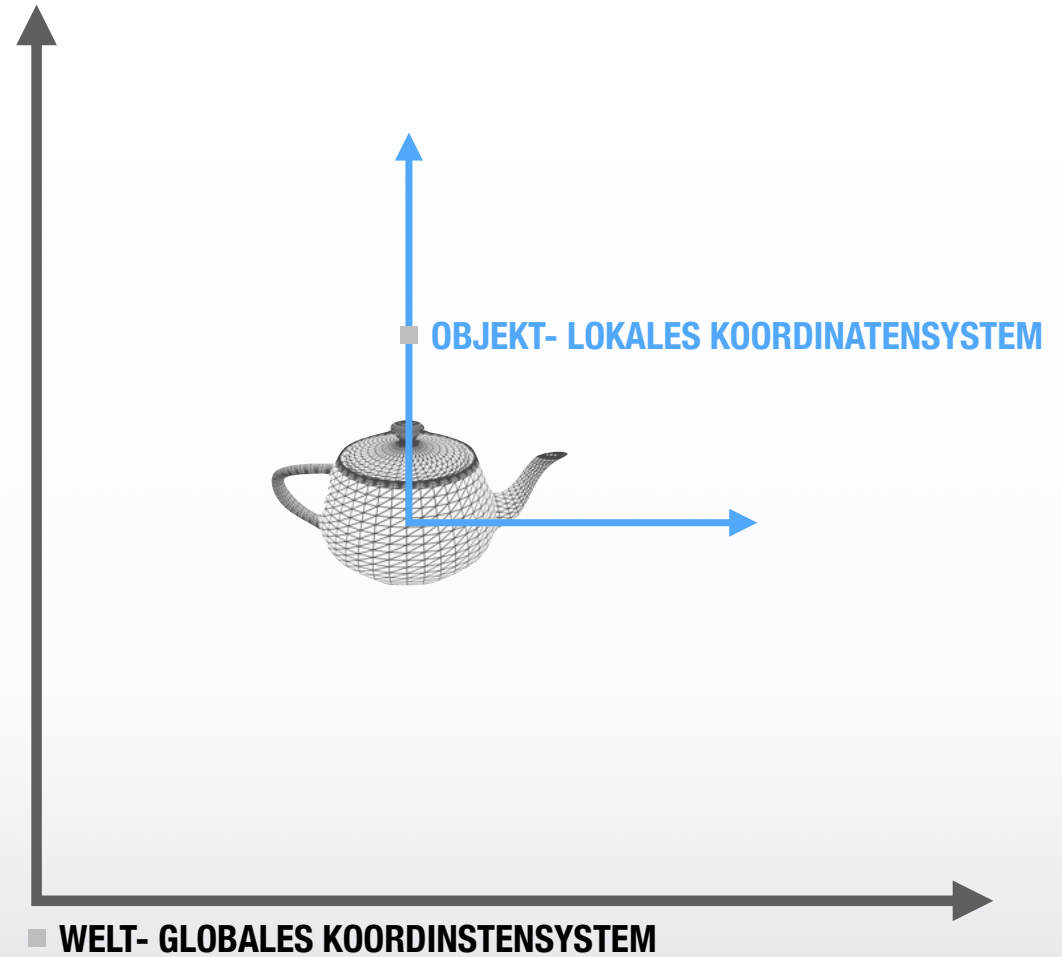


pipeline



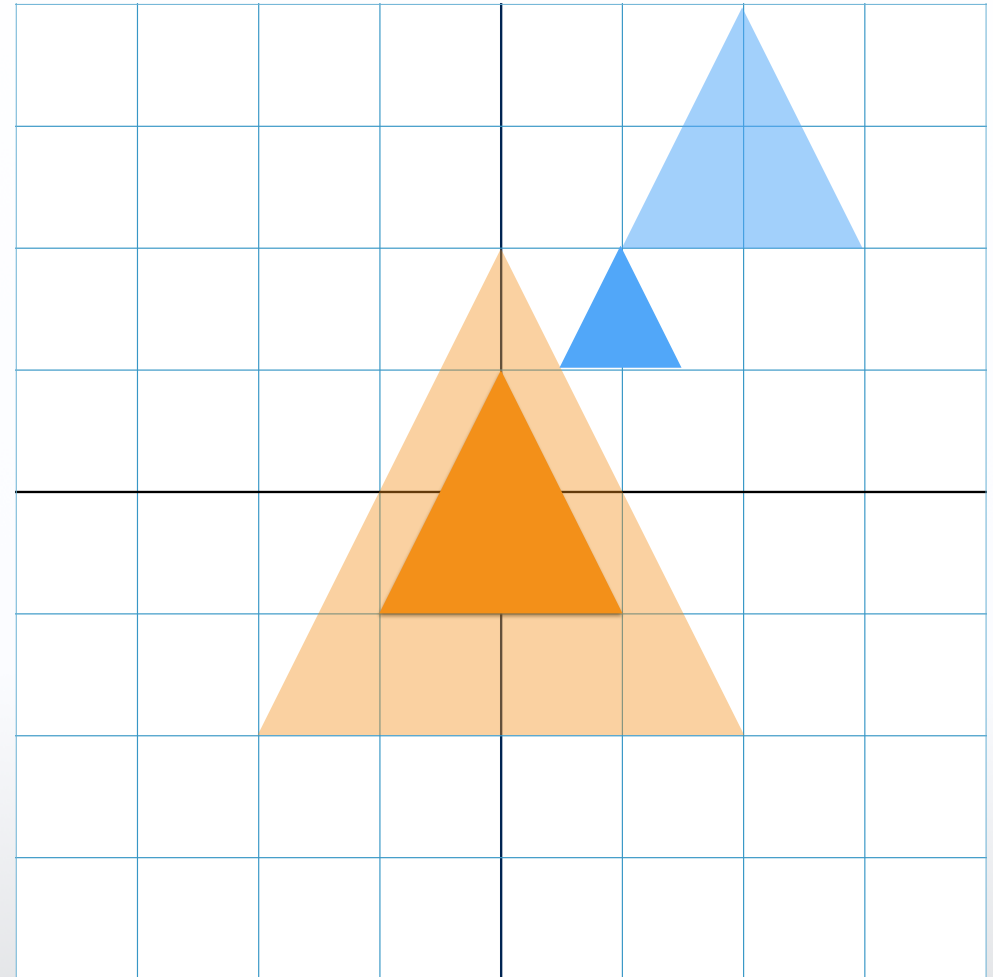
Wichtiger Begriff

- **OBJEKT- LOKALES KOORDINATENSYSTEM**
- **WELT- GLOBALES KOORDINATENSYSTEM**



Skalierung

- Durch vorherige Definition werden alle Punkte bis auf den Ursprung verschoben
- Der Abstand jedes Punktes vom Fixpunkt (Ursprung) ändert sich entsprechend der Skalierungsfaktoren
- Jedes Objekt hat ein lokales Koordinatensystem
- Ursprung des lokalen Koordinatensystems des Objektes beeinflusst das Ergebnis der Skalierung



2D Rotation

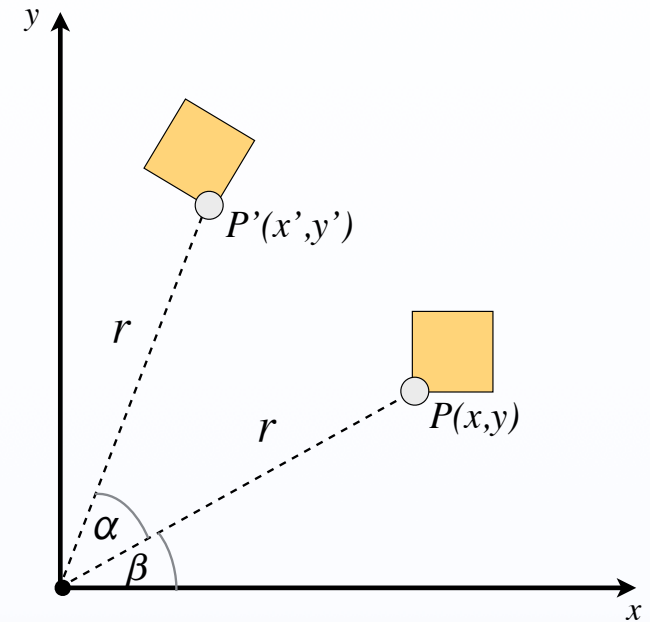
$$x' = x \cdot \cos \alpha - y \cdot \sin \alpha$$

$$y' = x \cdot \sin \alpha + y \cdot \cos \alpha$$

in Matrixschreibweise

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

oder $p' = R \cdot p$



Rotation um 90°



$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

Konkatenation von Matrizen

- $p = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$ Spaltenvektor !

- Beispiel 1: Translation, dann Rotation

- $p' = T \cdot p$
- $p'' = R \cdot p'$
- $p'' = R \cdot T \cdot p$

```
glRotatef(50, 0, 0, 1);  
glTranslatef(3, 0, 0);
```

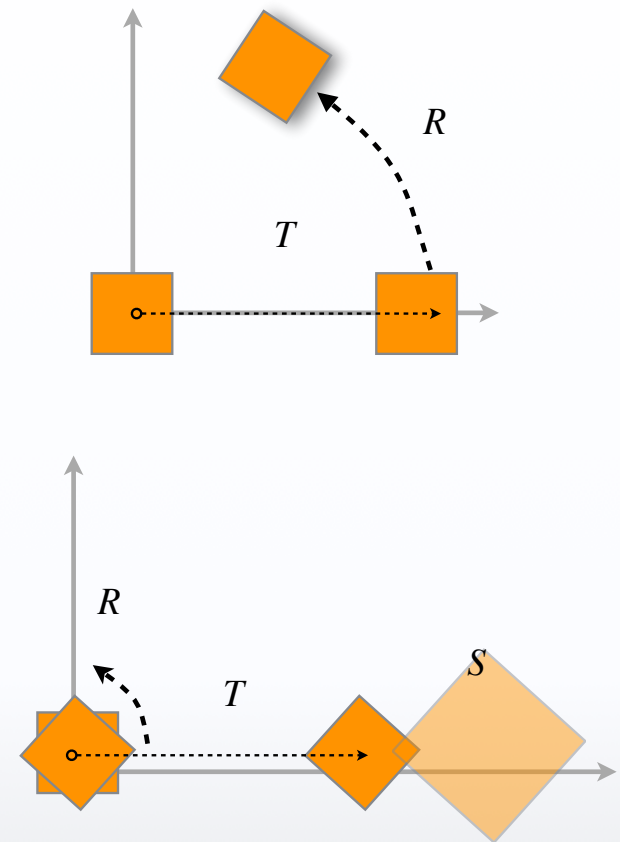
- Reihenfolge der Notation und der Ausführung sind entgegengesetzt !

- Beispiel 2:

- 1) Rotation $p' = R \cdot p$
- 2) Translation $p'' = T \cdot p'$
- 3) Skalierung $p''' = S \cdot p''$

$$p''' = S \cdot T \cdot R \cdot p$$

Alle Vertices ($v_1, \dots, v_n$) werden mit einer vorher initialisierten Matrix M multipliziert.



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} t_x \\ t_y \end{bmatrix} + \begin{bmatrix} x \\ y \end{bmatrix}$$

Translation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

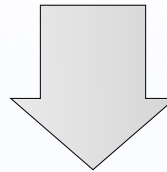
Skalierung

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Rotation

$$\begin{pmatrix} x + t_x \\ y + t_y \end{pmatrix} = \begin{pmatrix} t_x \\ t_y \end{pmatrix} + \begin{pmatrix} x \\ y \end{pmatrix}$$

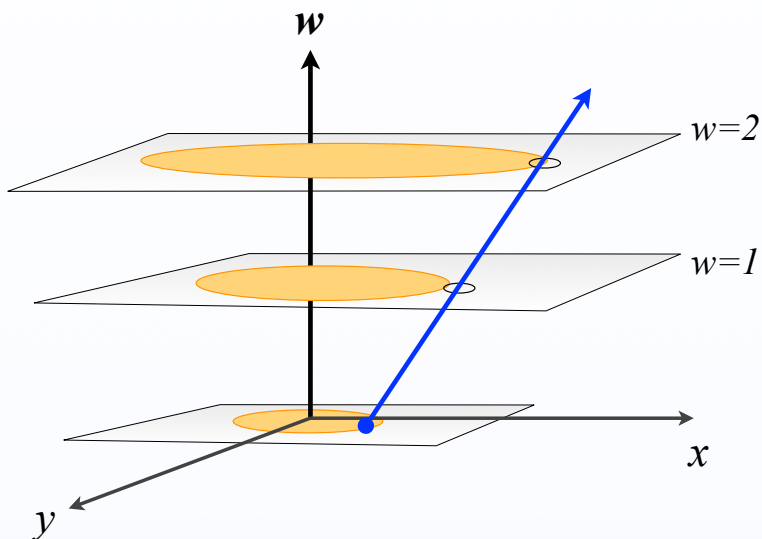
Darstellung durch Multiplikation



$$\begin{pmatrix} x + t_x \\ y + t_y \\ \textcircled{1} \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ \textcircled{1} \end{pmatrix}$$

Homogene Koordinaten (Roberts 1965)

- Ein Punkt $P(x,y)$ wird in homogener Koordinaten als Tripel dargestellt



Erweitert um die homogene Komponente w

$$\begin{pmatrix} x & y \end{pmatrix} \rightarrow \begin{pmatrix} x_h & y_h & w \end{pmatrix}$$

$$x_h = x \cdot w$$

$$y_h = y \cdot w$$

Kartesische Koord.

Homogene Koord.

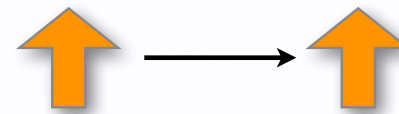
$$\begin{pmatrix} 2 \\ 1 \end{pmatrix} \rightarrow \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}_{w=1} \equiv \begin{pmatrix} 4 \\ 2 \\ 2 \end{pmatrix}_{w=2}$$

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Homogene Koordinaten

Translation

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$



Skalierung

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Rotation

$$R = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Transformation in 3D: 4X4 Matrix

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Translation um den Vektor:
Kommutativ!

$$t = (t_x, t_y, t_z)$$

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Skalierung in um den achselabhängige Faktor
Kommutativ!

$$s = (s_x, s_y, s_z)$$

$$R = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation um beliebigen Winkel um die Z-Achse
Kommutativ auf der selben Achse!

ACHTUNG bei mehreren Rotationen um mehrere Achsen!

Die meisten Transformationen in CG werden in homogenen Koordinaten durchgeführt. Erst am Ende (Malen des Punktes auf den Bildschirm) wird durch w dividiert

Transformation der Normale

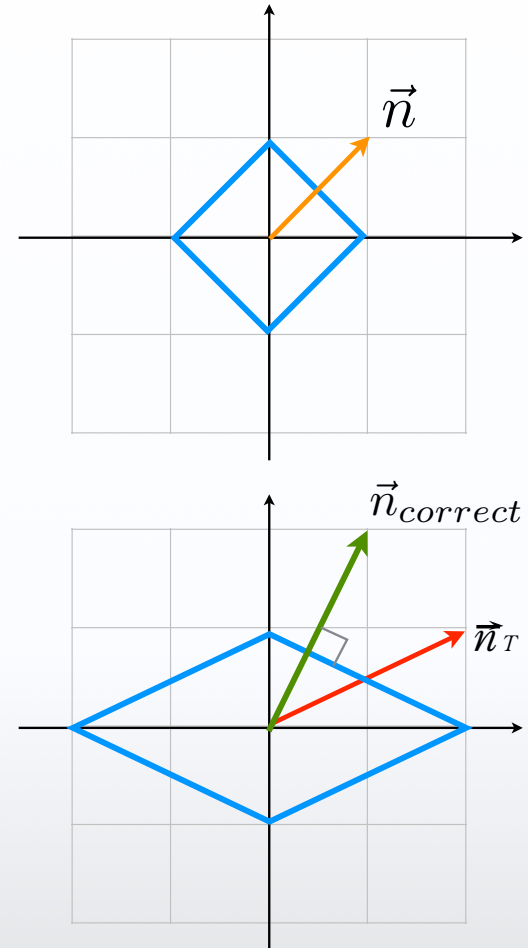
Die Normale auf der Fläche: $\vec{n} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix}$

Transformation lautet: $S = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Die transformierte Normale:

$$\vec{n}_T = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

Richtig wäre: $\vec{n}_{correct} = \begin{pmatrix} 1 \\ 2 \\ 0 \\ 0 \end{pmatrix}$



Transformation der Normale

Mathematisch korrekt ist $n_Q^T \cdot t_M = 0$ Skalarprodukt (Zeilenvektor • Spaltenvektor)

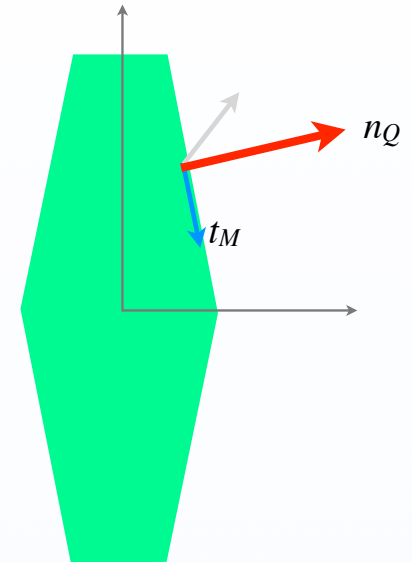
Und dies die korrekte Herleitung:

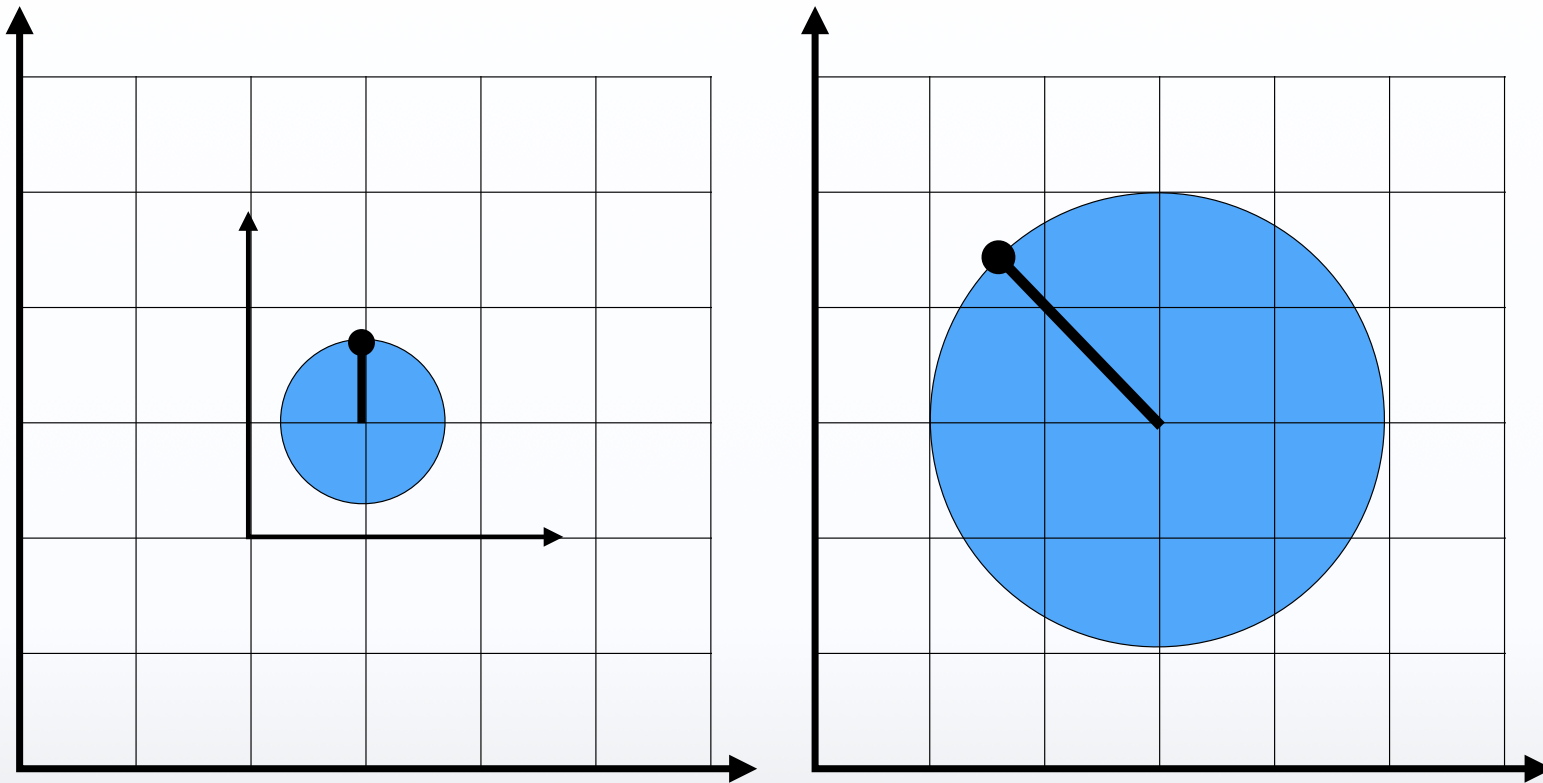
Gesucht ist die Transformation Q

$$n_Q^T = (n^T \cdot M^{-1})$$

$$\begin{aligned} n_Q &= (n^T \cdot M^{-1})^T \\ &= (M^{-1})^T \cdot n \end{aligned}$$

$$Q = (M^{-1})^T$$



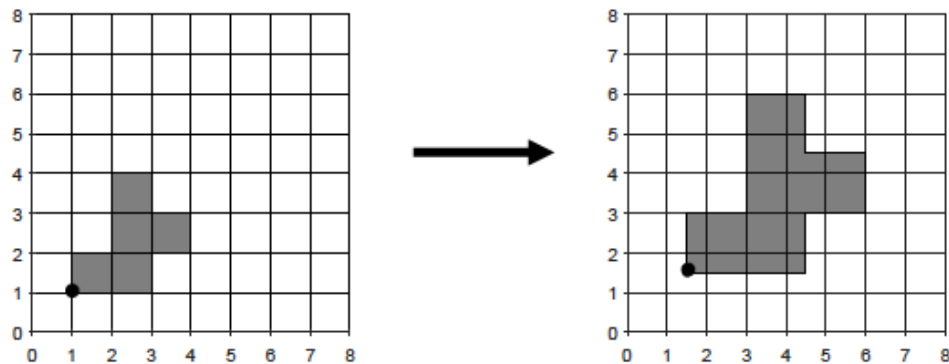


2. Transformationen

Die nachfolgenden Teilaufgaben zeigen links den Anfangs- und rechts den Endzustand der Szene. Füllen Sie die fehlenden Matrizen aus, die zu dem gewünschten Endzustand führen.

Jede Teil-Matrix darf ausschließlich eine Translations-, Skalierungs- oder Rotationsmatrix enthalten. Der Nullpunkt des lokalen Objektkoordinatensystems liegt genau in der Mitte des schwarzen Kreises. Wir arbeiten, wie aus der Vorlesung bekannt mit Spaltenvektoren.

2.1. Ein Objekt wird wie folgt transformiert:

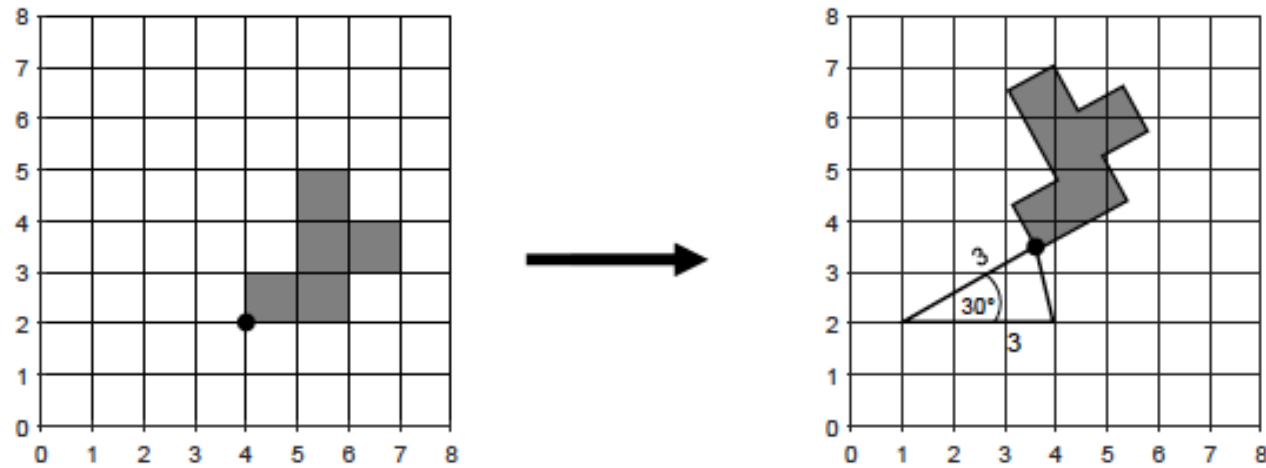


Bestimmen Sie M .

$$M = \begin{pmatrix} & \\ & \end{pmatrix}$$

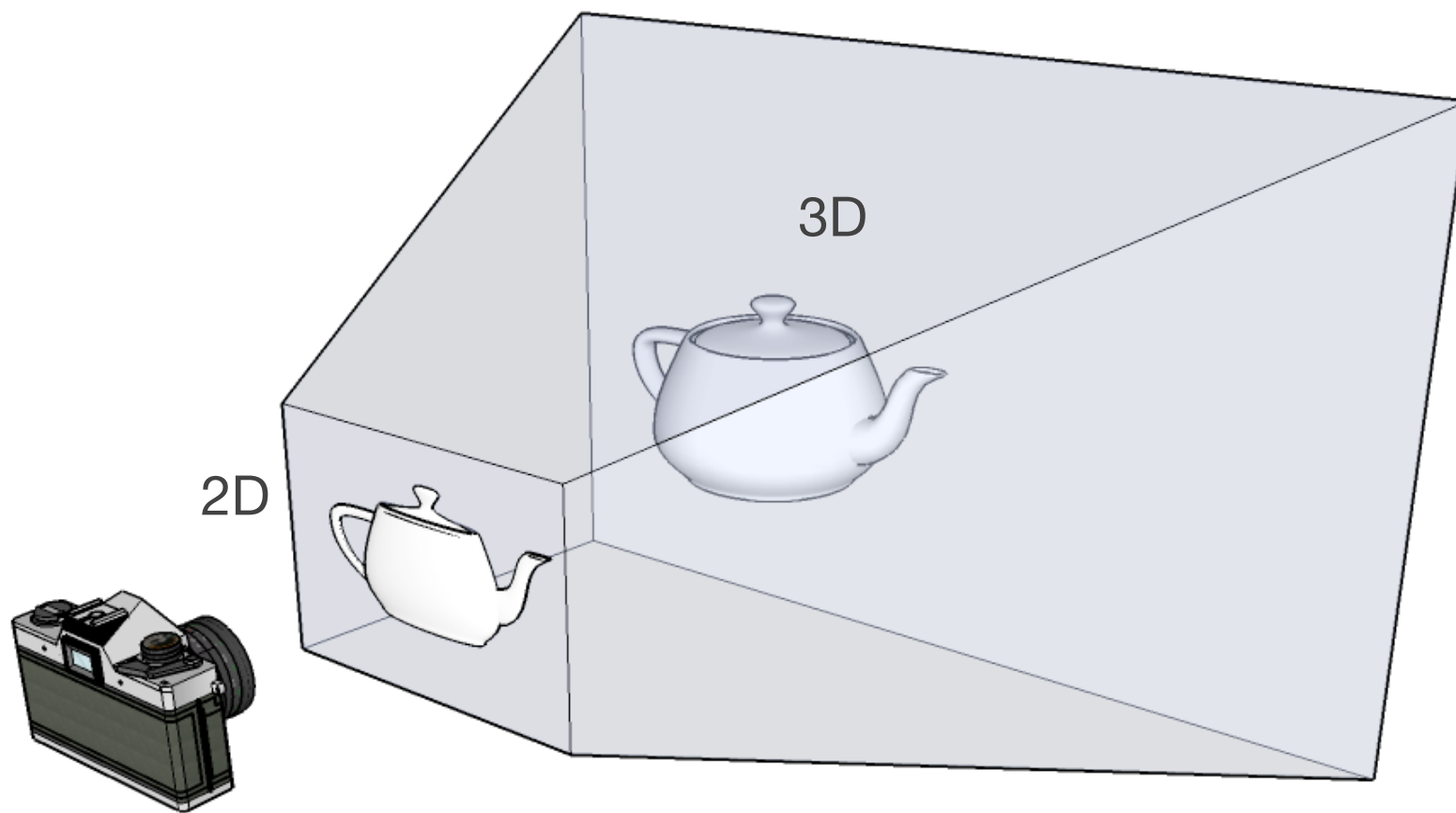
2.4. Ein Objekt wird wie folgt transformiert:

Wichtig: Es muss um den Punkt (1 ; 2) gedreht werden.

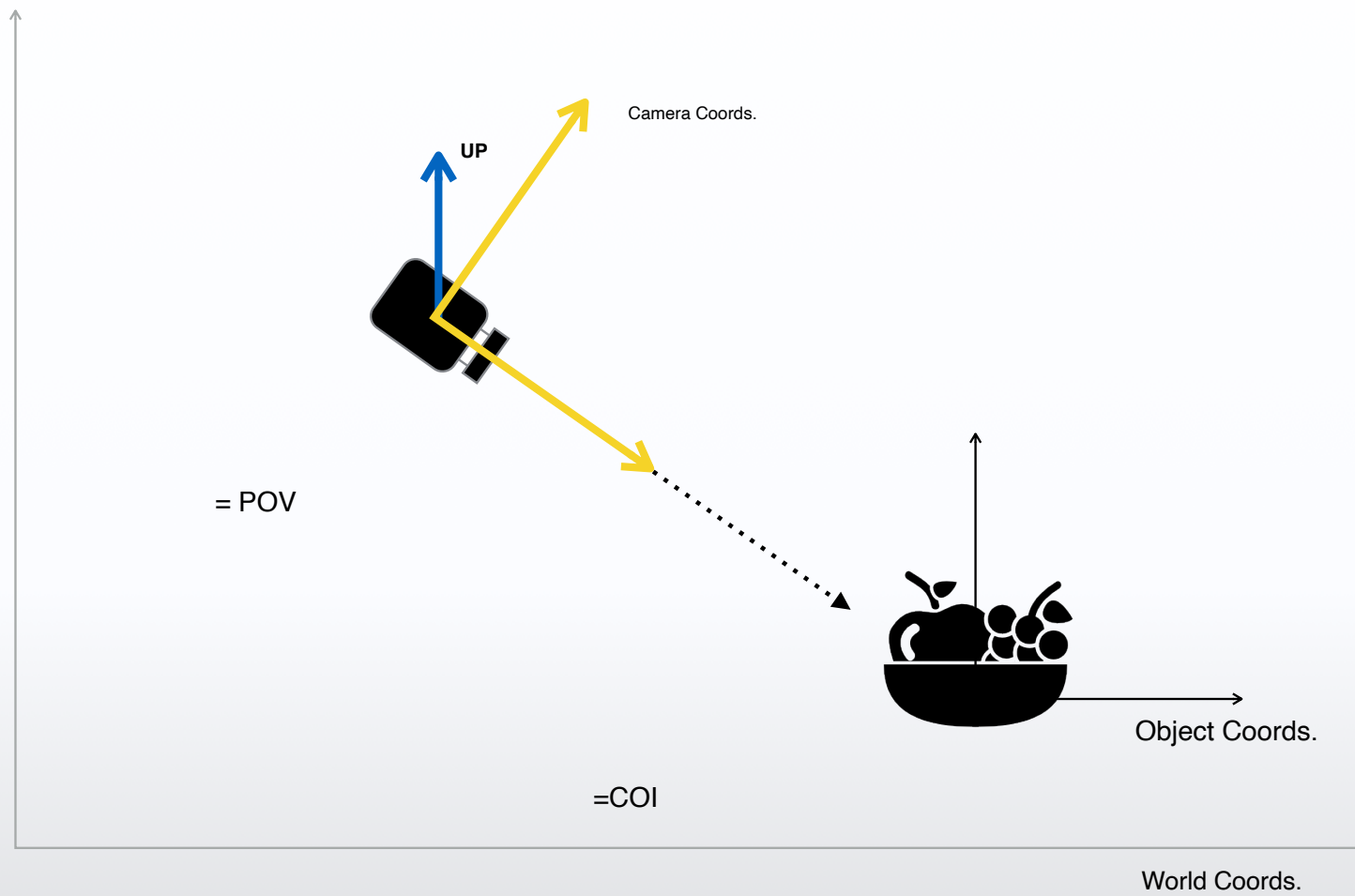


Bestimmen Sie M .

$$M = \begin{pmatrix} & \end{pmatrix} \cdot \begin{pmatrix} & \end{pmatrix} \cdot \begin{pmatrix} & \end{pmatrix}$$



Viewpoint



View Coordinates

■ gegeben:

- Viewpoint: P_0
- Center of Interest C (Center)
- Up Vector $\vec{g}$

■ gesucht:

- Camera Coord.sys. $(\vec{v}, \vec{u}, \vec{s})$ (lokales Kamerakoord.sys)

?

Wie erzeugen wir den View-Vektor?

$$\vec{v} = C - P_0$$

?

Wie erzeugen wir den Side-Vektor?

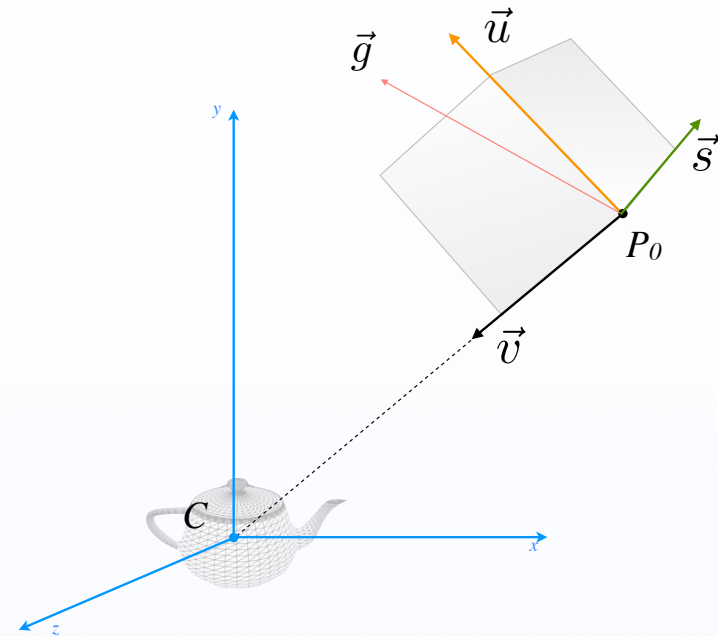
$$\vec{s} = \vec{v} \times \vec{g}$$

?

Wie erzeugen wir den neuen Up Vektor

$$\vec{u} = \vec{s} \times \vec{v}$$

Alle Vektoren normieren



Koordinaten Transformation

- Tatsächlich wird die gesamte Szene mit der Kamera transformiert.

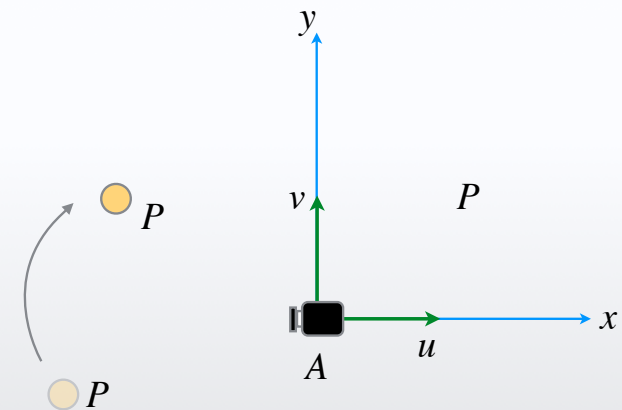
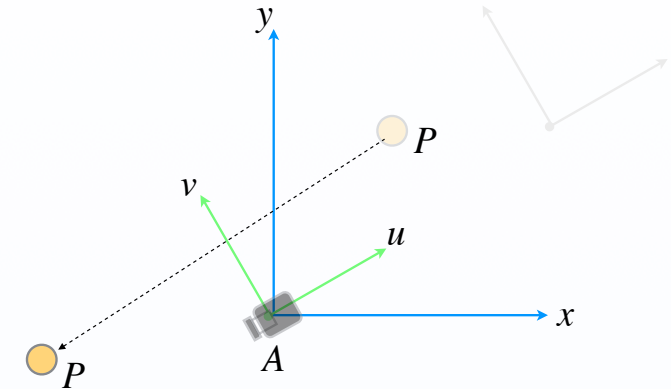
$$T = \begin{pmatrix} 1 & 0 & -A_x \\ 0 & 1 & -A_y \\ 0 & 0 & 1 \end{pmatrix}$$

- Rotation, so dass die Achsen übereinstimmen.

$$R = \begin{pmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Wir bestimmen die spezielle Projektionsmatrix gültig für

- Im Ursprung
- Achsenparallel
- in Richtung der negativen Z-Achse



Wozu das Ganze ?

A) Formel der völlig freien Projektion zu aufwendig. (folgt später)

Voraussetzungen schaffen um die Formel zu vereinfachen

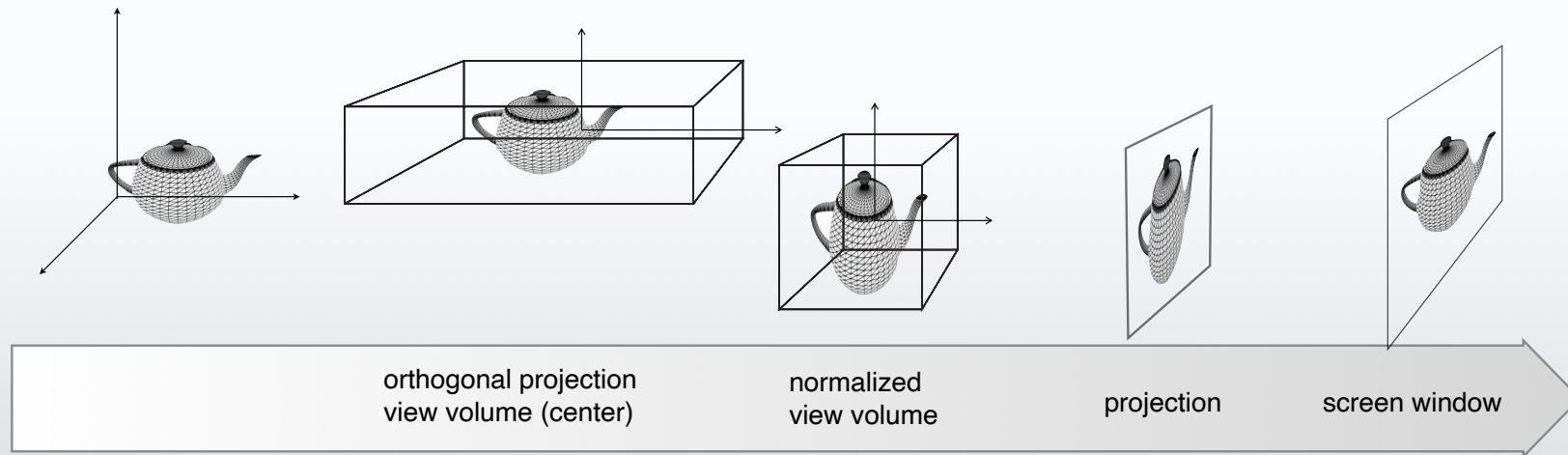
1. Kamera ist im Nullpunkt
2. Kamera ist achsenparallel ausgerichtet
3. Kamera blickt in Richtung der negative Z-Achse

B) Clipping funktioniert nur mit achsenparallelen Ebenen.

C) BackFaceCulling ist viel einfacher (*nur Vorzeichen z-Achse*)

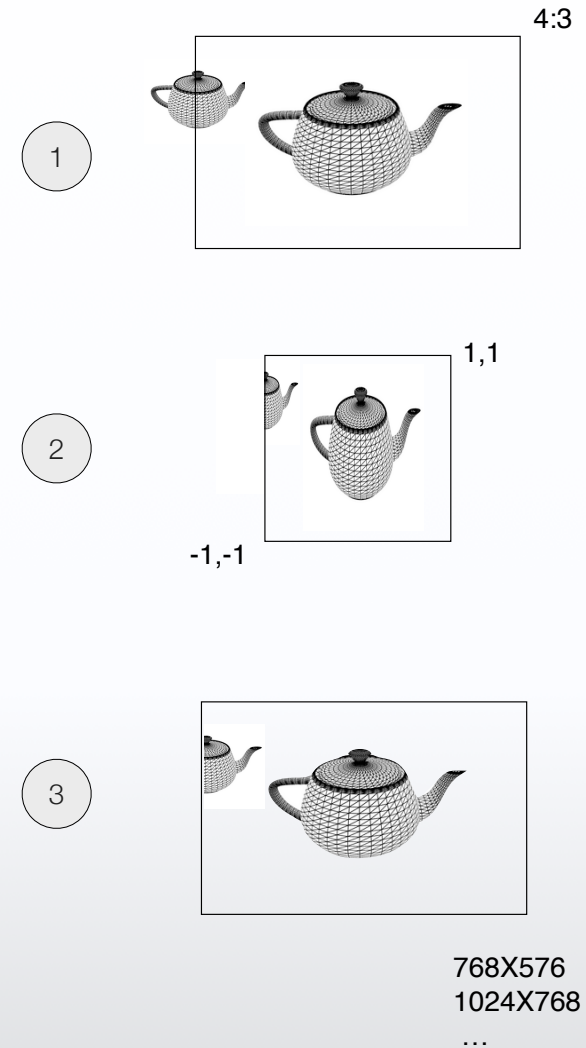
Normalization for Orthogonal Projection 3D

- Position im Weltkoordinatensystem
- Position im Kamerasystem
- Forderung: Orthogonale Projektion
- Viewpoint & -Volume festlegen
- View Volume zentrieren
- View Volume normalisieren
- Back Face Culling
- Clipping - mit achsenparallelen Ebenen (+1,-1)
- Eine Koordinate eliminieren (Projektion)
 - Tiefenwert für Verdeckung nutzen
- View Port Transformation
 - Position in Screen Koordinatensystem



Warum Normierung ?

- Orthogonal Projection View Volume
- Normierung auf Einheitsvolumen $(-1,1)$
- **Culling** (Back Face Removal)
 - Testen ob der Z Wert einen negativen Wert hat
- **Clipping** der Polygone
 - Schnittpunkte und Clipping-Verfahren gegen Achsenparallele Ebenen
 - Alle Gleichungen und Berechnungen einfacher, wenn die Ebenen mit $-1,+1$ zu definieren sind.
- **Viewporttransformation**
 - „Entzerren“ des „Sichtquadrates“ auf die endgültige Fenstergröße des Sichtfensters
 - Mit den normierten Werten können die Werte einfach zum „Pixel-Wert“ umgerechnet werden.



`glOrtho(left,right,bottom,top,near,far)`

$$M = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{l+r}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Normalization Transform

Die Konkatenation der Transformationen:

$$M = \begin{matrix} \text{Skalierung} & & \text{Translation} & & \text{Spiegelung} \\ \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} & \cdot & \begin{pmatrix} 1 & 0 & 0 & -\frac{l+r}{2} \\ 0 & 1 & 0 & -\frac{b+t}{2} \\ 0 & 0 & 1 & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix} & \cdot & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{matrix}$$

$\xleftarrow{\quad M_{\text{Ortho}} \quad} \quad \xrightarrow{\quad M_{R \sim L} \quad}$

$$M = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{l+r}{2} \\ 0 & \frac{2}{t-b} & 0 & -\frac{b+t}{2} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

`glOrtho(left,right,bottom,top,near,far)`

Perspective Projection in OpenGL

```
void gluPerspective(GLdouble fov, GLdouble aspect, GLdouble near, GLdouble far);
```

$$M = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{l+r}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{b+t}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

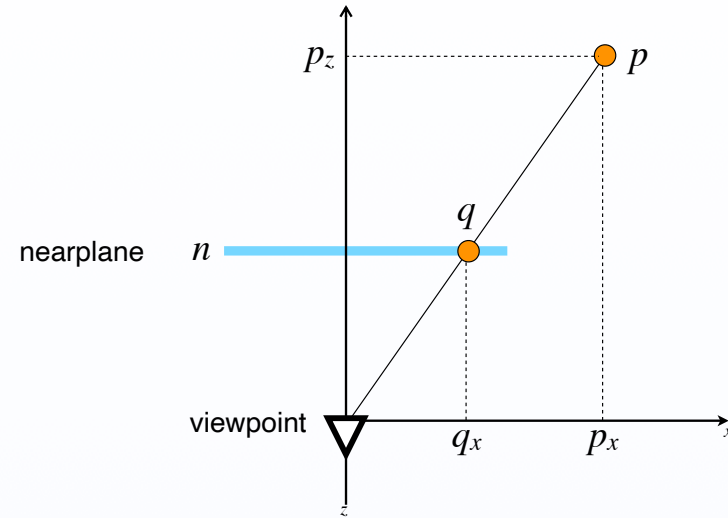


Perspective Projection

1 Strahlensatz

$$\frac{q_x}{p_x} = \frac{n}{p_z} \rightarrow q_x = n \cdot \frac{p_x}{p_z}$$

$$\frac{q_y}{p_y} = \frac{n}{p_z} \rightarrow q_y = n \cdot \frac{p_y}{p_z}$$



2 Wie kriegen wir das in einer 4X4 Matrix rein

Perspective Projection

$$\frac{q_x}{p_x} = \frac{n}{p_z} \rightarrow q_x = n \cdot \frac{p_x}{p_z}$$

$$\frac{q_y}{p_y} = \frac{n}{p_z} \rightarrow q_y = n \cdot \frac{p_y}{p_z}$$

$$\begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} n \cdot p_x \\ n \cdot p_y \\ n \cdot p_z \\ p_z \end{pmatrix} \xrightarrow{\text{homog.}} \frac{1}{p_z} \rightarrow \begin{pmatrix} (n \cdot p_x)/p_z \\ (n \cdot p_y)/p_z \\ n \\ 1 \end{pmatrix}$$

! PROBLEM ?

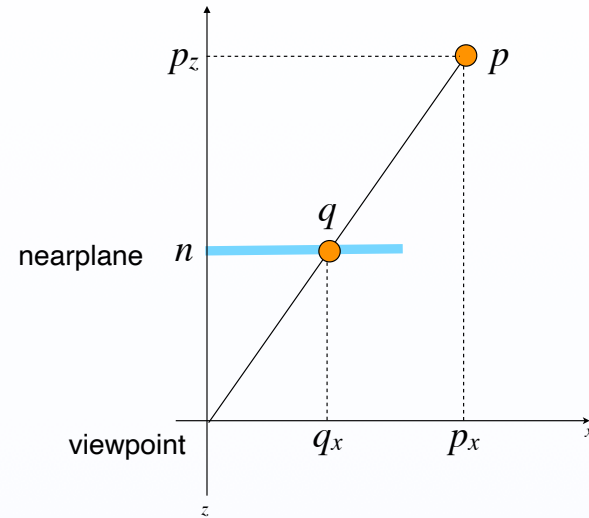
Perspective Projection

- Was passiert mit den z Werten
- Integrierbar in einer 4x4 Matrix
- Tiefeninformation (Sichtbarkeit) soll nicht verlorengehen
- Ansatz:

$$\begin{pmatrix} q_x \\ q_y \\ q_z \\ 1 \end{pmatrix} \rightarrow \begin{pmatrix} n \cdot p_x / p_z \\ n \cdot p_y / p_z \\ ? \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} q_x \\ q_y \\ q_z \\ 1 \end{pmatrix} \rightarrow \begin{pmatrix} np_x / p_z \\ np_y / p_z \\ \sim p_z \\ 1 \end{pmatrix}$$

Ein Wert welcher Information über die Tiefe enthält.



Perspective Projection

- Die z Werte müssen folgende Bedingungen erfüllen
 - Die near Werte (n) werden auf near (n) abgebildet
 - Die far Werte (f) werden auf far (f) abgebildet
 - Alle z Werte dazwischen behalten ihre Ordnung (Monoton steigend) / Proportional p_z
- Alles muss in einer 4X4 Matrix integrierbar sein

$$\frac{n \cdot f}{p_z} \begin{cases} p_z = n \rightarrow f \\ p_z = f \rightarrow n \end{cases} \rightarrow (n + f) - \frac{n \cdot f}{p_z}$$

$$\begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n + f & -n \cdot f \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} n \cdot p_x \\ n \cdot p_y \\ (n + f) \cdot p_z - n \cdot f \\ p_z \end{pmatrix}$$

- Lokales Koordinatensystem
- Vertex (x,y,z,w) / Normalen (x,y,z,w) / Farbe (r,g,b,a) / Material (amb. / diffuse / spec.) / Textur (u,v) / ...
- Objekte in das globale Koordinatensystem transformieren
- Normalen mit der transponierten Inversen Matrix
- Kamera aufstellen (Position/ lookat/ up-vector)
- Transformation in das Kamerakoordinatensystem
- Auge im Ursprung / Blick entlang der negativen Z-Achse / Rechtssystem
- Beleuchtung, Illumination (Phong,...)
- Projektionstransformation
- Rechts ins Linkssystem Transformation vom Persp. Frustum in einem Quader
- Quader skalieren & zentrieren & auf -1/+1
- Culling / Clipping
- z-Werte für die Verdeckung benutzen (Z-Buffer)
- Projektion (Division durch w) (ab hier 2D)
- Viewport Transformation
- x,y Werte (-1...+1) werden auf das View Window (Pixel) abgebildet
- Rastering
- Shading
- Texturing

3D

Object Coordinates

Global Coordinates

Modelview Matrix

View Coordinates

Projection Matrix

Clip Coordinates

Perspective Division

2D

Normalized device Coord.

Viewport Transformation

Window Coordinates

6. Perspektivische Projektion

6.1. Aus welchen Benutzerangaben wird das Kamerakoordinatensystem erstellt?



6.2. Aus welchen Gründen muss aus diesen Angaben zunächst ein orthogonales Kamerakoordinatensystem erstellt werden?

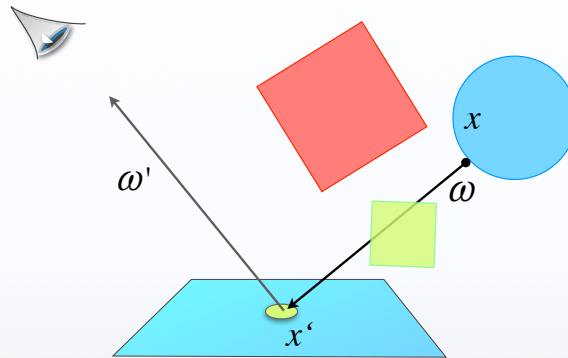






Illumination

$$L(x', \omega') = E(x', \omega') + \int \rho(\omega, \omega') \cdot L(x, \omega) \cdot G(x, x') \cdot V(x, x') dA$$



1. Einschränkungen der Lichtquellen

1. Nur beschränkte Anzahl von primären Punktlichtquellen
2. Ambiente Raumbelichtung wird approximiert durch eine Konstante

2. Keine Anisotropische Reflexion

1. Nicht alle Eingang- und Ausgangswinkel sondern,
2. Reflexion und Refraktion finden nur in einer Ebene statt

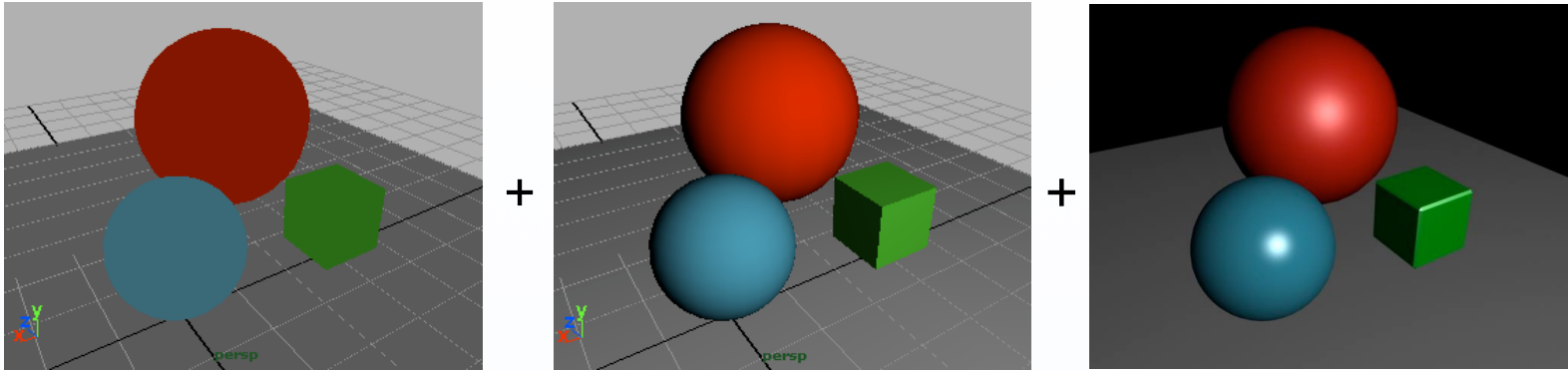
3. Licht wird nur genau einmal reflektiert

4. Approximation der BRDF durch 3 Anteile

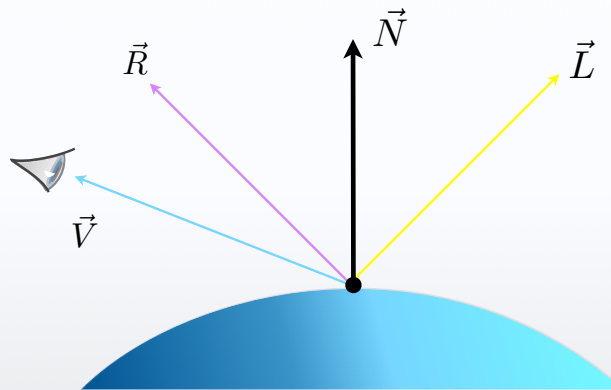
1. ambient
2. diffus
3. spekulär

$$L(x', \omega') = E(x', \omega') + \int \rho(\omega, \omega') L(x, \omega) G(x, x') V(x, x') dA$$

Phong Beleuchtungsmodell



$$i_{ges} = i_a + i_d + i_s$$



$$I = I_a \cdot k_a + \sum_{p=0}^m I_p \cdot [k_d(\overline{N} \cdot \overline{L}) + k_s(\overline{V} \cdot \overline{R})^n]$$

$$I = I_a \cdot k_a + \sum_{p=0}^m I_p \cdot [k_d(\overline{N} \cdot \overline{L}) + k_s(\overline{V} \cdot \overline{R})^n]$$

Komponentenweise

$$\begin{pmatrix} I_R \\ I_G \\ I_B \end{pmatrix} = \begin{pmatrix} I_{aR} \\ I_{aG} \\ I_{aB} \end{pmatrix} \cdot \begin{pmatrix} k_{aR} \\ k_{aG} \\ k_{aB} \end{pmatrix} + \sum_{p=0}^m \begin{pmatrix} I_{pR} \\ I_{pG} \\ I_{pB} \end{pmatrix} \cdot \left[\begin{pmatrix} k_{dR} \\ k_{dG} \\ k_{dB} \end{pmatrix} \cdot \left(\begin{pmatrix} \vec{N}_x \\ \vec{N}_y \\ \vec{N}_z \end{pmatrix} \circ \begin{pmatrix} \vec{L}_x \\ \vec{L}_y \\ \vec{L}_z \end{pmatrix} \right) + \begin{pmatrix} k_{sR} \\ k_{sG} \\ k_{sB} \end{pmatrix} \cdot \left(\begin{pmatrix} \vec{V}_x \\ \vec{V}_y \\ \vec{V}_z \end{pmatrix} \circ \begin{pmatrix} \vec{R}_x \\ \vec{R}_y \\ \vec{R}_z \end{pmatrix} \right)^n \right]$$

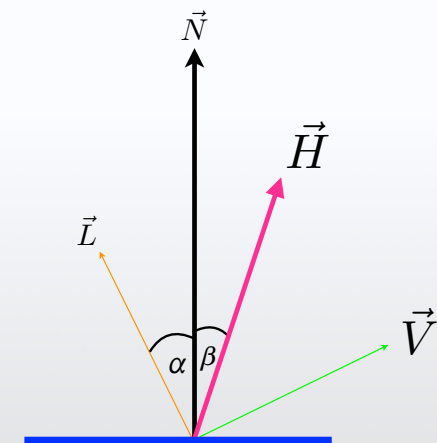
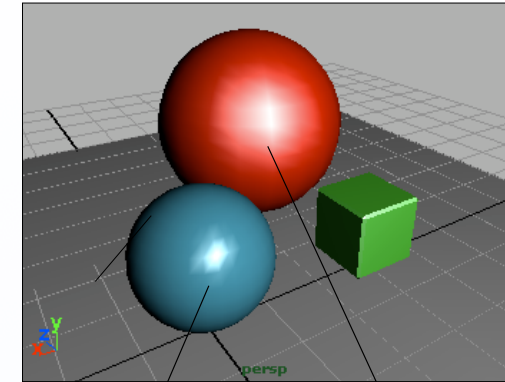
Skalarprodukt (*norm* (0..1))

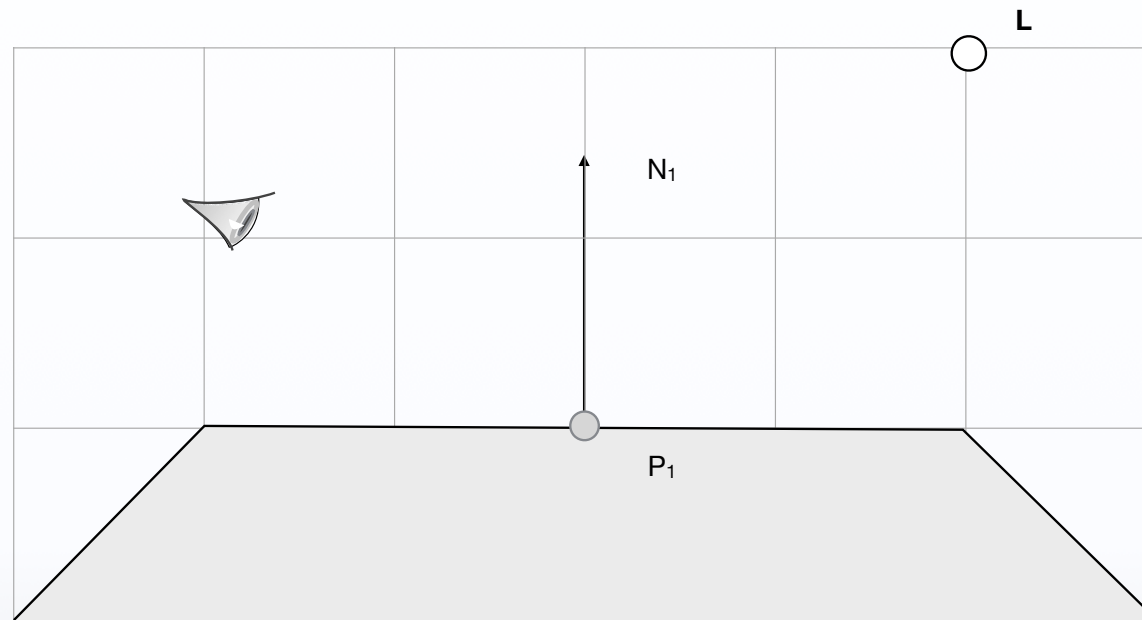
Blinn Illumination

- Variante der Phong-Beleuchtung
- Blinn:
 - Halbvektor zwischen Lichtvektor und dem View-Vektor
- $\vec{H} = \vec{L} + \vec{V}$, nicht normiert

$$i_{ges} = I_a \cdot k_a + \sum_{p=0}^m I_p \cdot [k_d(\vec{N} \cdot \vec{L}) + k_s(\vec{V} \cdot \vec{R})^n]$$

$$i_{ges} = I_a \cdot k_a + \sum_{p=0} I_p \cdot [k_d(\vec{N} \cdot \vec{L}) + k_s(\vec{N} \cdot \vec{H})^n]$$



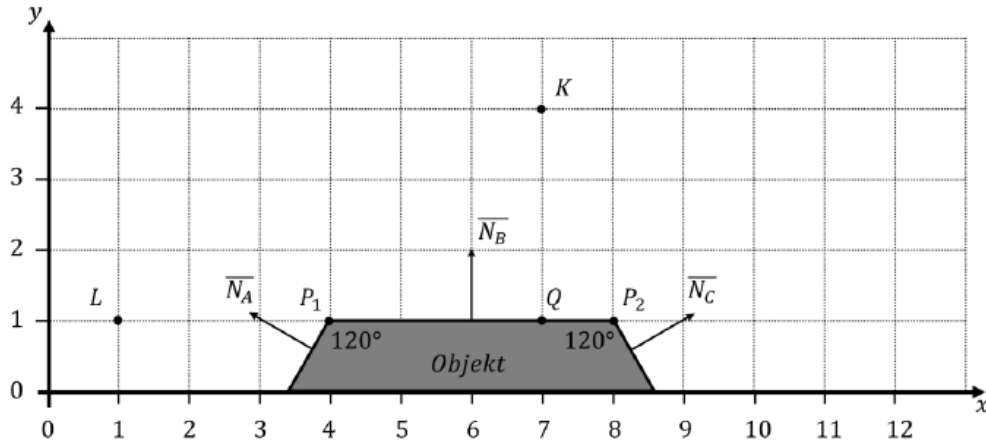


Geg.: $L, N_1, k_a, k_d, k_s,$

Ges.: I an der Stelle P_1

4. Beleuchtung und Shading 2

Gegeben sei folgende Szene:



Kameraposition: $K = (7, 4)^T$
 Lichtposition: $L = (1, 1)^T$
 Vertices: P_1, P_2
 Flächennormalen: $\overline{N}_A, \overline{N}_B, \overline{N}_C$
 Materialeigenschaften: $k_a, k_d, k_s = \{1/2 ; 1/2 ; 1/2\} \quad n = 2$
 Intensität von L: $I_L = \{1/2 ; 1/2 ; 1/2\}$
 Ambiente Licht Intensität: $I_a = \{1/8 ; 1/8 ; 1/8\}$

Hinweis: Farben, Intensitäten und Reflexionskoeffizienten sind für die drei Farbkanäle rot (r), grün (g) und blau (b) angegeben als $\{r ; g ; b\}$.

4.1. Es sei $\overline{N}_1$ die normierte Eck-Normale am Vertex P_1 und $\overline{N}_2$ die normierte Eck-Normale am Vertex P_2 . Zeigen Sie, dass folgendes gilt:

$$\overline{N}_1 = \left(-\frac{1}{2}, \frac{\sqrt{3}}{2}\right)^T \quad \overline{N}_2 = \left(+\frac{1}{2}, \frac{\sqrt{3}}{2}\right)^T$$

4.2. Die Szene wird mit Phong-Illumination gerendert. Es sei C_1 die Farbe am Vertex P_1 und C_2 die Farbe am Vertex P_2 .

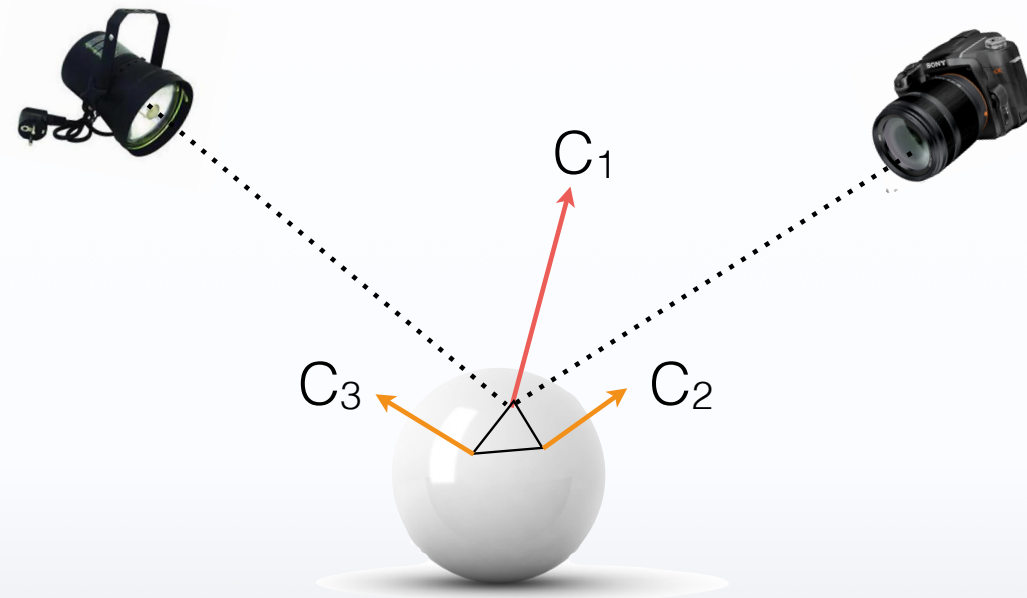
Zeigen Sie, dass gilt:

$$C_1 = \{0,4208; 0,4208; 0,4208\} \quad C_2 = \{0,0625; 0,0625; 0,0625\}$$

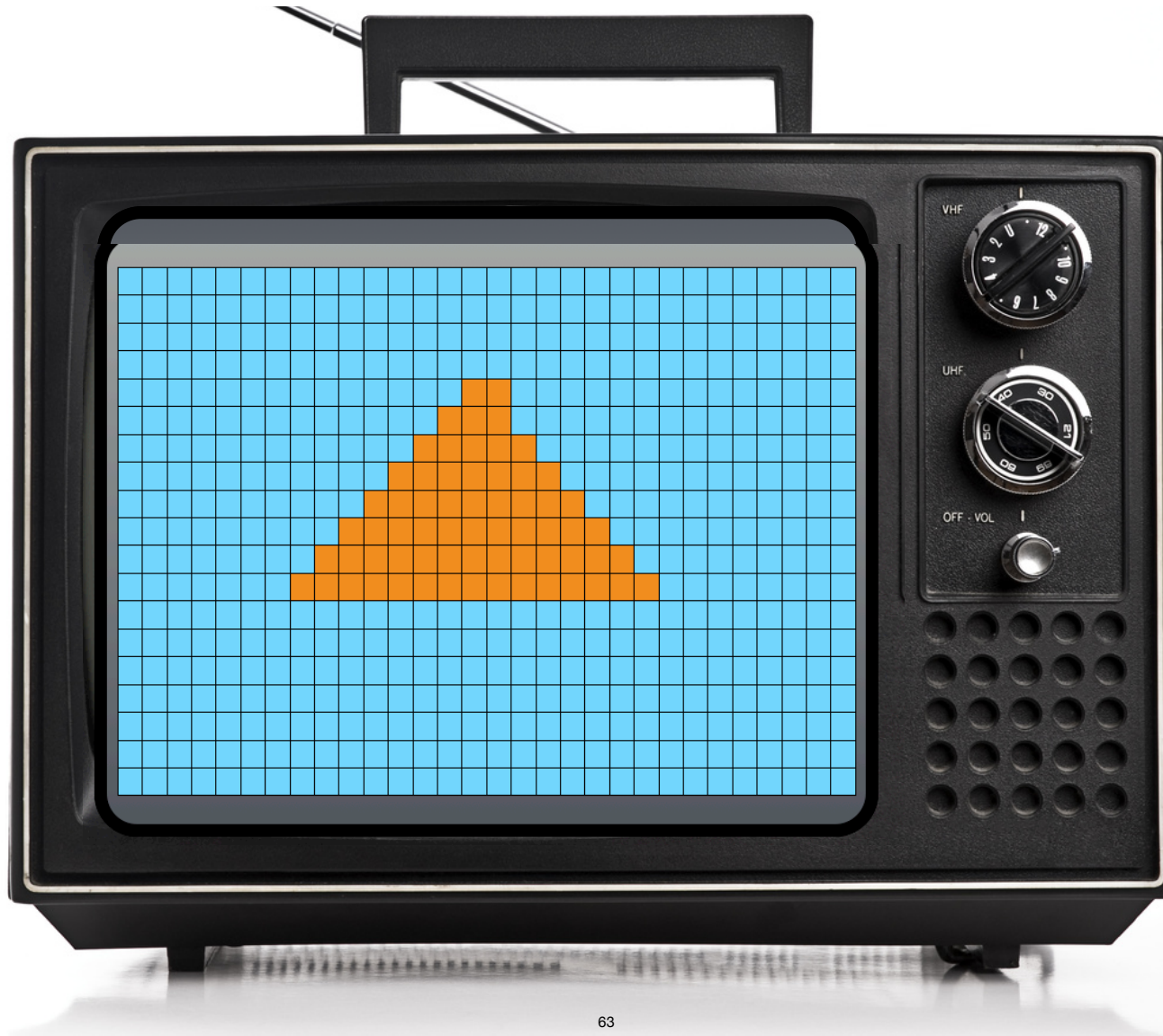
Hinweis: Benutzen Sie für Ihre Berechnungen die Eck-Normalen aus Unterpunkt 4.1. Runden Sie lediglich Ihre Endergebnisse auf vier Nachkommastellen.

SHADING

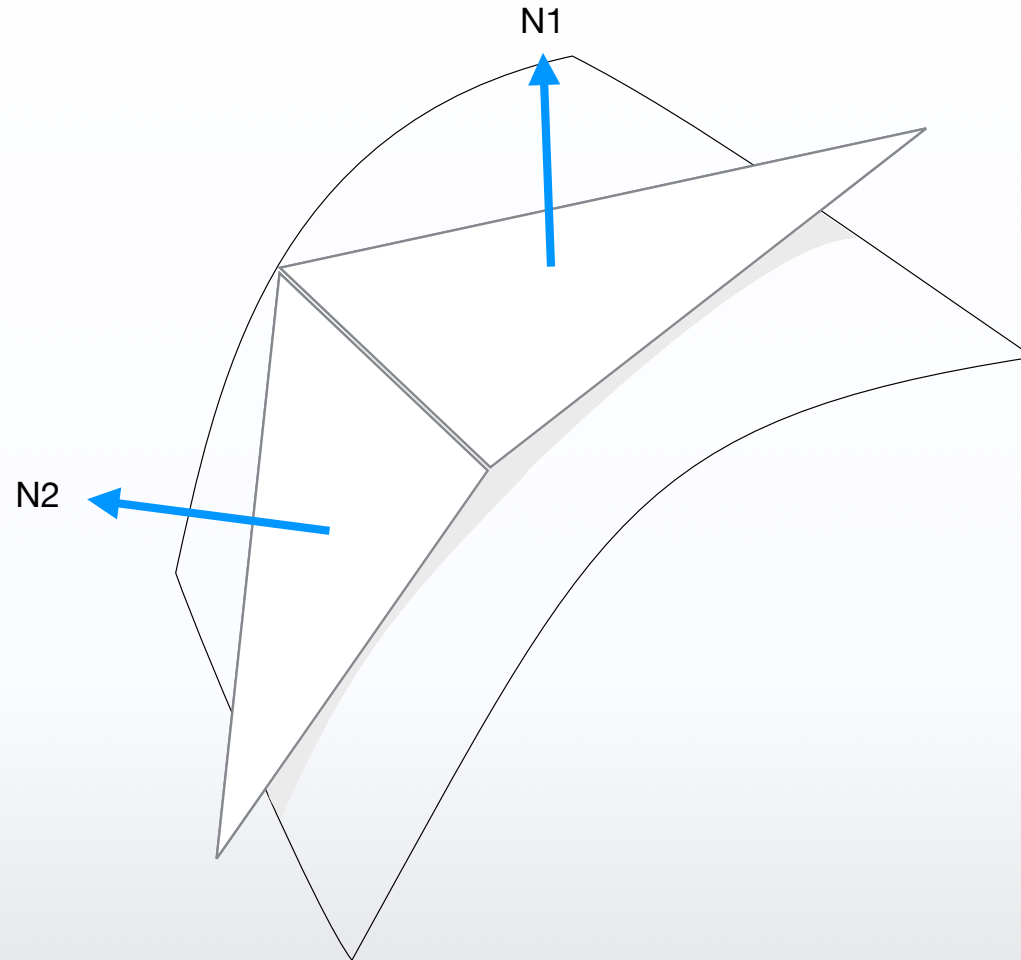
Ein Polygon: Drei Eckwerte, Drei Frabwerte

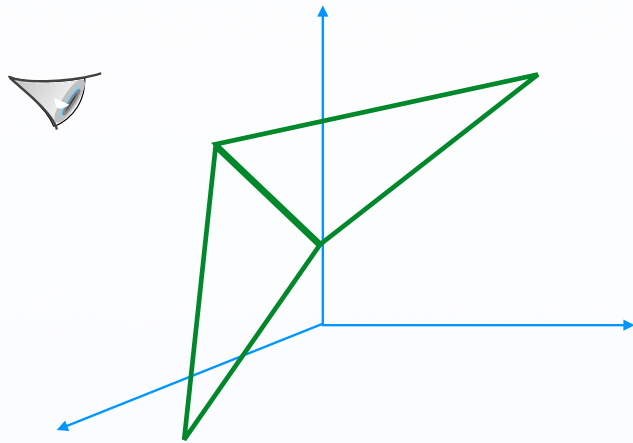


Shading 2D • after projection









PROJECTION



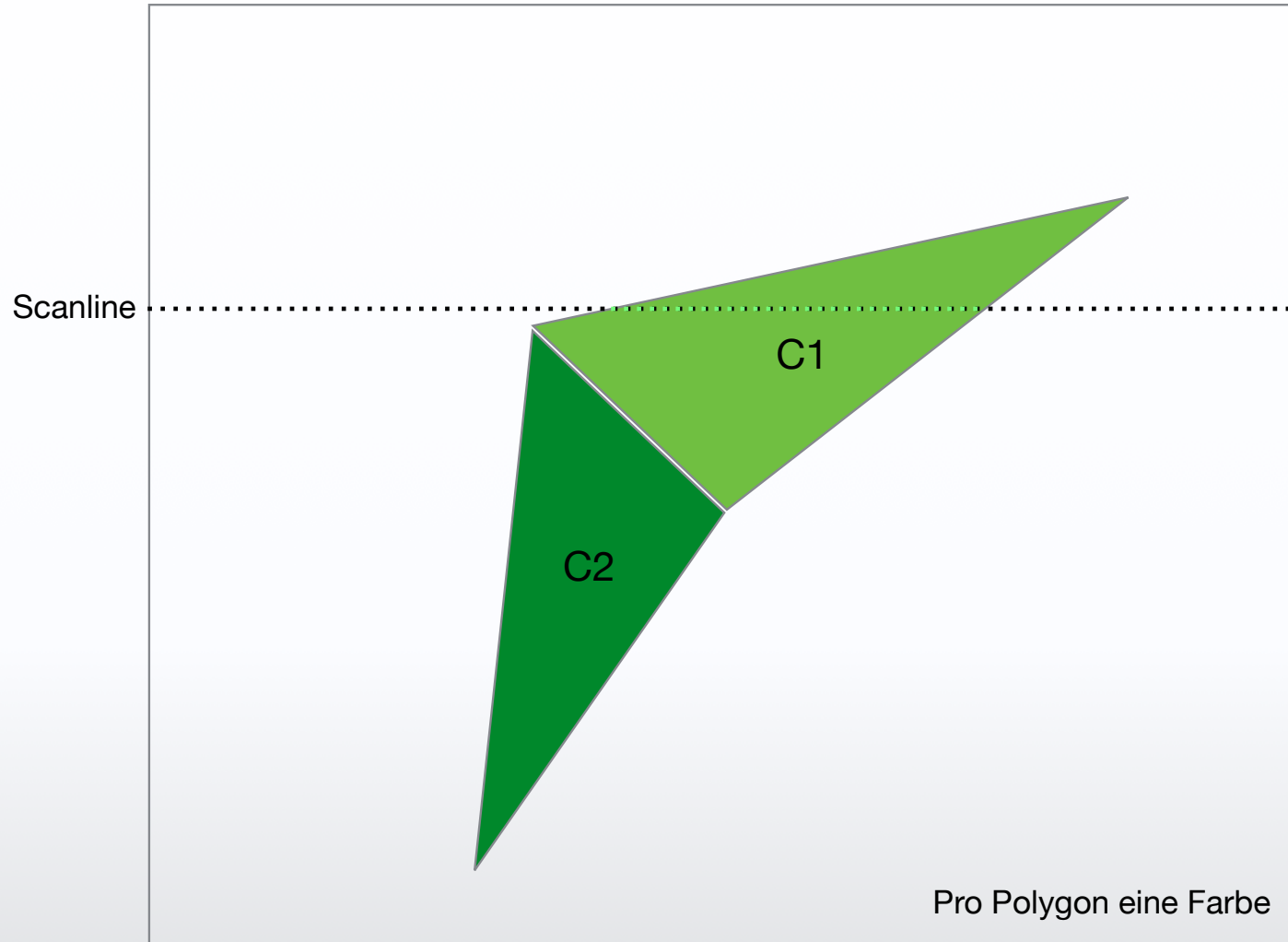
World

Vertexfarbwerte

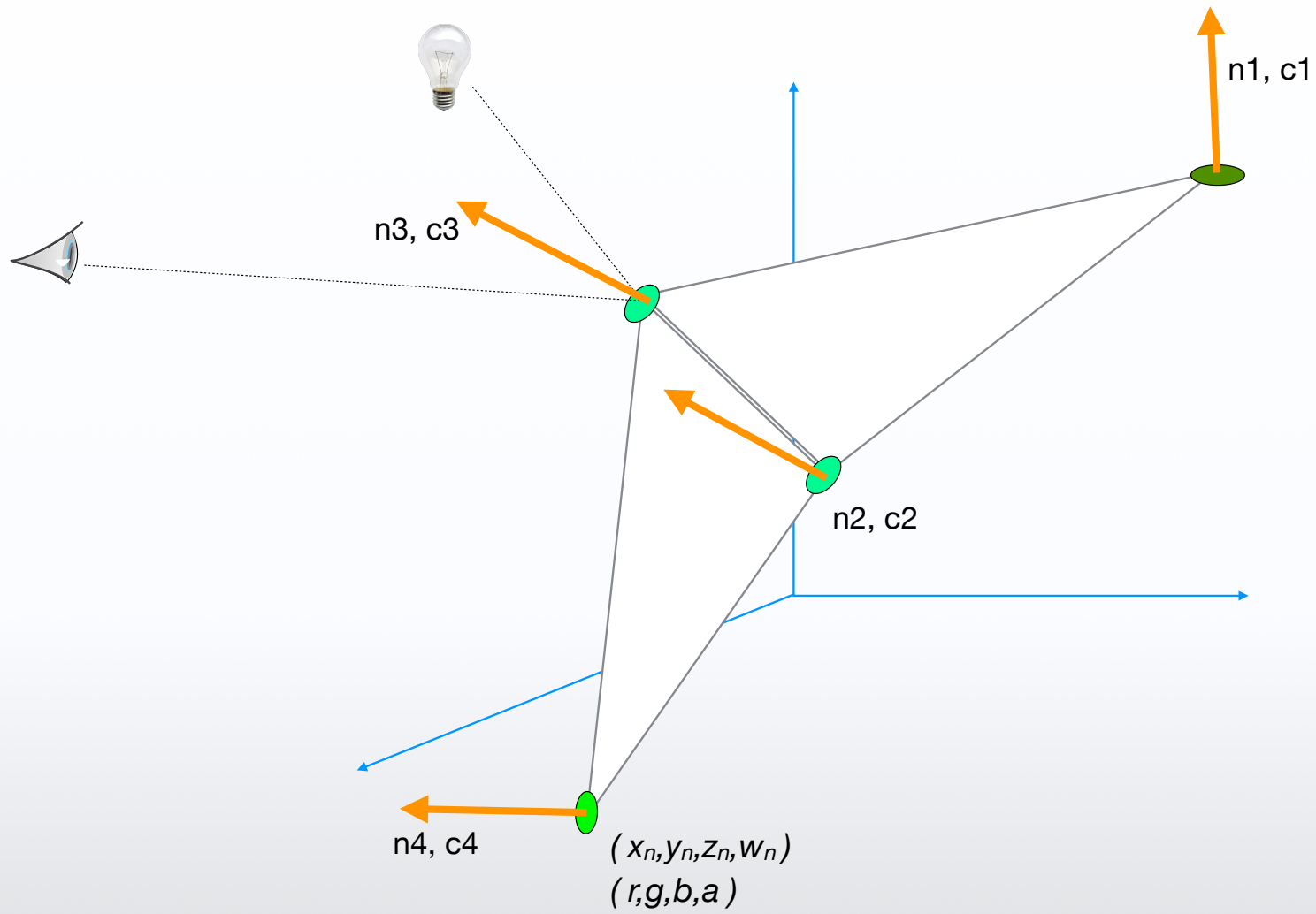
Window

Fläche füllen (Interpolation)

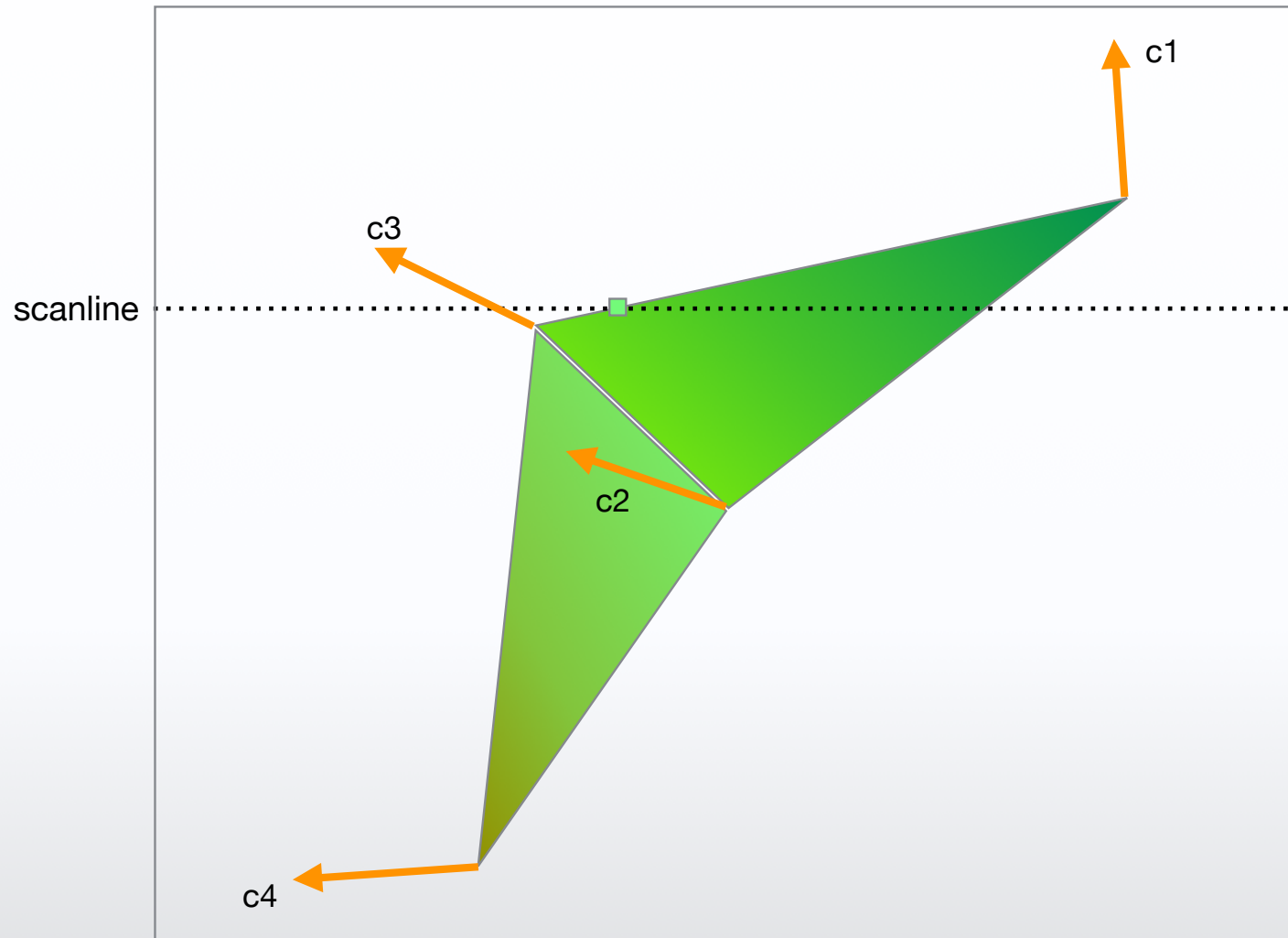
Flat shading



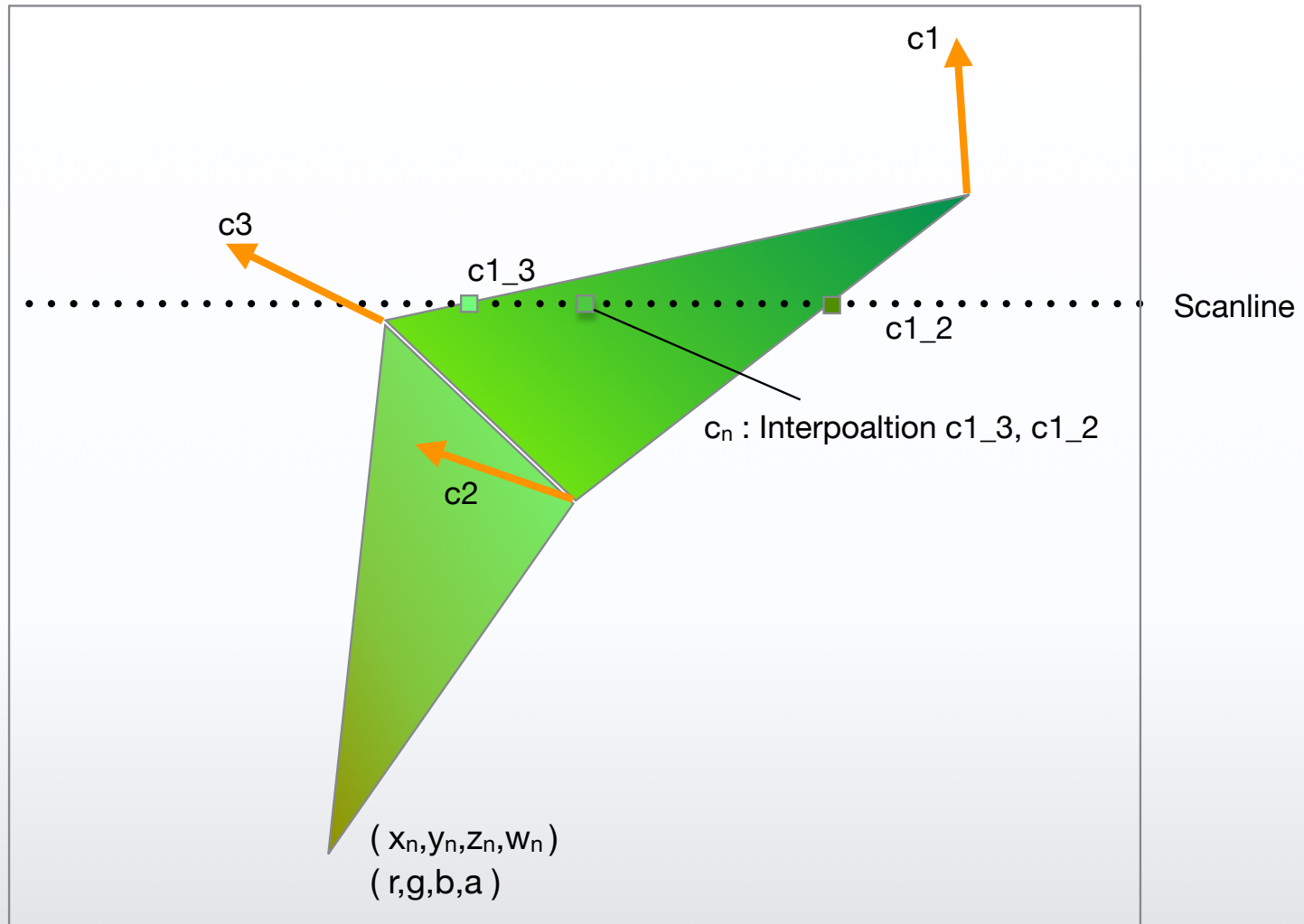
Vertex Illumination



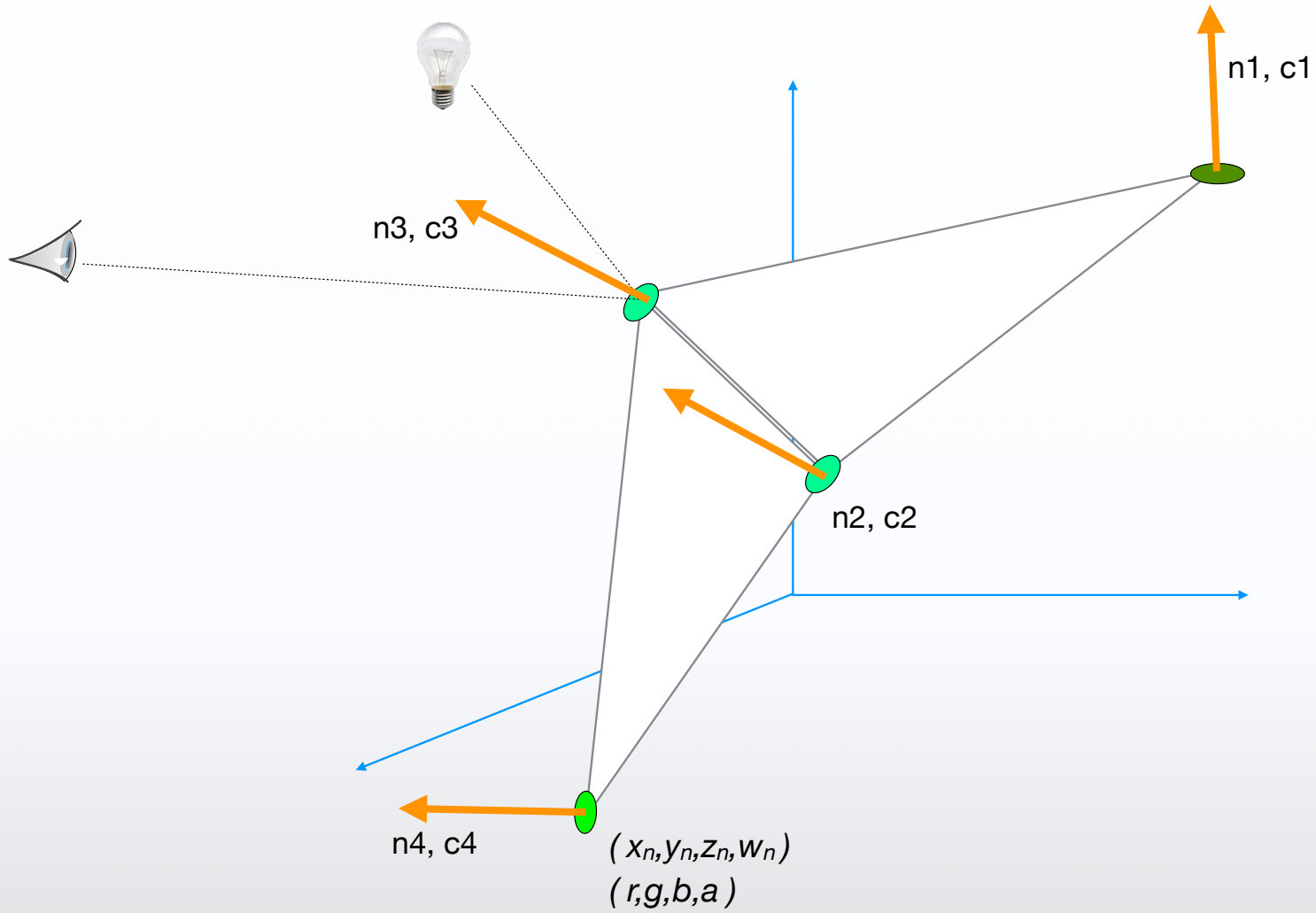
Gouraud Shading - color interpolation



Gouraud Shading - colorinterpolation

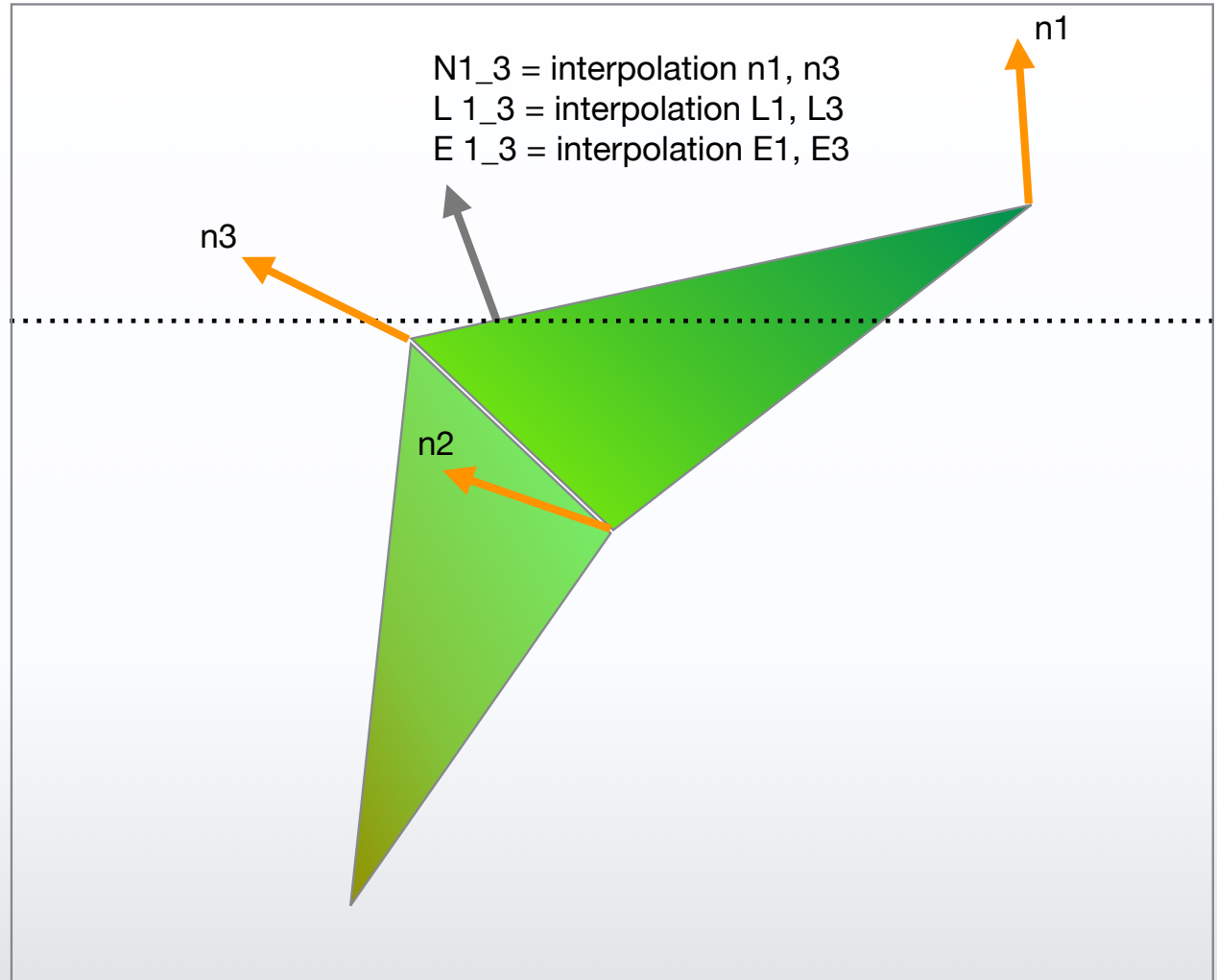


Vertex Illumination

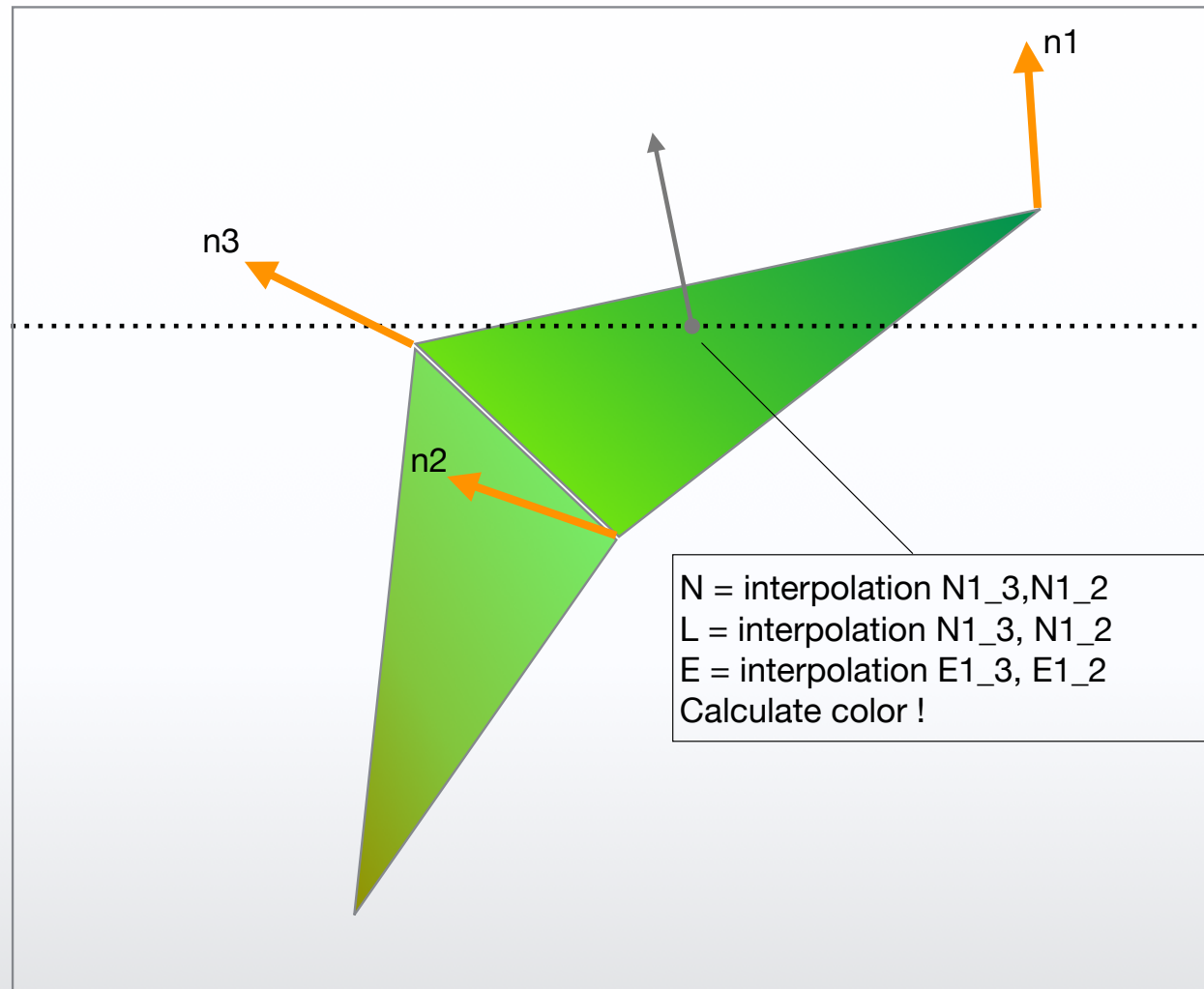


Phong Shading - normal interpolation

Alle Vektoren werden interpoliert
Am aktuellen Punkt wird die
Illumination neu berechnet



Phong Shading - normal interpolation

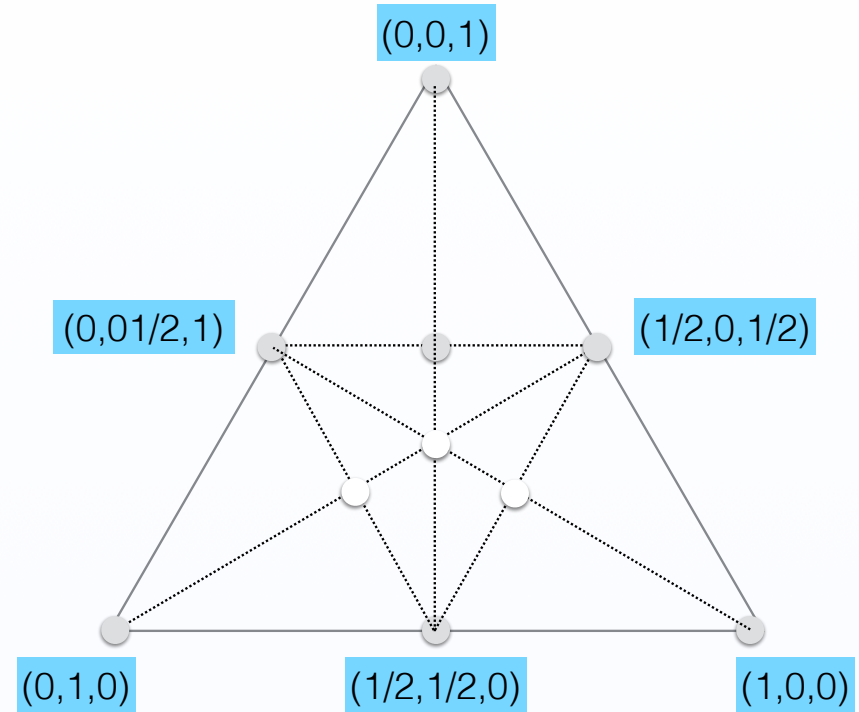


Baryzentrische Koordinaten

- Jeder Punkt p lässt sich bestimmen als:

$$p(\alpha, \beta, \gamma) = \alpha a + \beta b + \gamma c$$

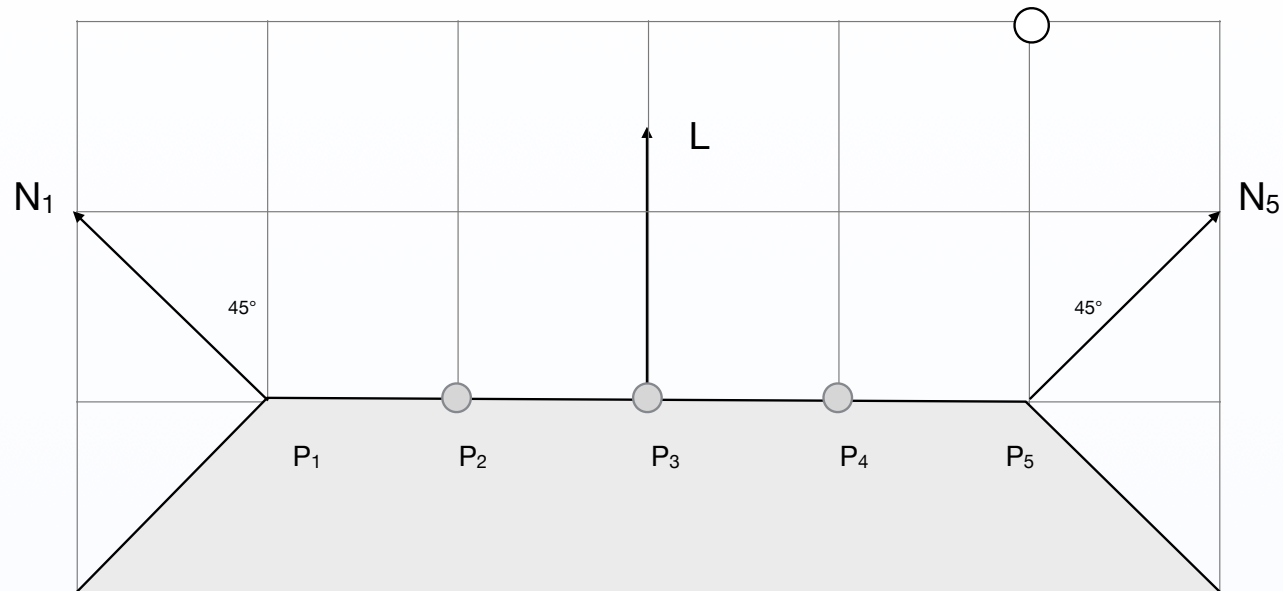
mit $\alpha \equiv 1 - \beta - \gamma$



Innerhalb
wenn alle Koordinaten
zwischen $(0, \dots, 1)$ (Inside/
Outside-Test)

Kante (Edge)
Eine Koordinate 0
Die anderen zwischen $(0, \dots, 1)$

Ecke (Vertex)
Zwei Koordinaten 0,
Einer ist 1



Geg.: L , N_1 , k_a , k_d , k_s ,

Ges.: I an der Stelle P_3

/ Gouraudshading bei ..

/ Phongshading bei ..

- 8.1. Beschreiben Sie das Phong-Shading Verfahren. Unter Berücksichtigung der Farbwerte, Normalenwerte, Einordnung in der Pipeline.
- 8.2. Erklären Sie die Unterschiede für die drei Beleuchtungsmodelle Flat Shading, Gouraud Shading und Phong Shading. Wie unterscheiden sich diese drei Modelle hinsichtlich der Auswertung des Normalenvektors, der Betrachtungsrichtung, der Beleuchtungsrichtung? Welche Vorteile und Nachteile haben die Verfahren?